Universidad Andrés Bello®

Facultad de Ingeniería
Desarrollo Web y Móvil
Evaluación de la Unidad 2: **Diseño e implementación de un backend seguro completo**
Caso N.° 15: **Mr. Sandwich**, venta de sándwiches, hamburguesas, promociones y bebidas con retiro o delivery

Campo	Datos	Campo	Datos
Integrantes	Max Fernández, Tomás Hurtado	Fecha	09-11-2026
Docente	Ernesto Vivanco	NRC	8478
Versión del frontend	Commit 6aa8c51 del 26-10-2026, carpeta *proyecto backend y front end/frontend*	Repositorio	https://github.com/Makkusu88/Desarrollo-web-y-movil
Ejecución: el sistema completo se inicia con `python run_all.py -reset`. Las instrucciones, dependencias, pruebas y evidencias están en el repositorio.			

Índice

Introducción 2
- Objetivo general . . . 2
- Objetivos específicos . . . 2
- Requisitos del caso . . . 2
- Brechas declaradas . . . 2

Módulo 1 Servicios derivados del frontend 3
- 1.1 Catálogo de servicios . . . 4
- 1.2 Ejemplos de contratos con datos ficticios . . . 7

Módulo 2 Clases y cardinalidades del dominio 7
- 2.1 Relaciones y reglas . . . 9
- 2.2 Decisiones del equipo . . . 9
- 2.3 Operación UML, ID de servicio . . . 9

Módulo 3 Sistemas e integraciones 9
- 3.1 Matriz de sistemas . . . 12
- 3.2 Intercambios entre sistemas . . . 13

Módulo 4 Stakeholders y permisos HTTP 13
- 4.1 Registro de stakeholders . . . 13
- 4.2 Matriz de permisos . . . 13

Módulo 5 Clases del API Gateway 14

Módulo 6 Rutas y secretos de integración 16
- 6.1 Matriz de rutas . . . 16
- 6.2 Operaciones GraphQL . . . 17
- 6.3 Métodos gRPC . . . 17
- 6.4 Inventario simbólico de secretos e identidades . . . 18

Módulo 7 Datos sensibles y secretos del cliente 19
- 7.1 Matriz de datos . . . 20
- 7.2 Decisión sobre secretos del cliente . . . 21
- 7.3 Sesiones y tokens . . . 21

7.4 Recorrido de un dato restringido: la dirección de entrega . . . 21
7.5 Casos de validación ejecutados . . . 21

Módulo 8 Separación privada y agregaciones GraphQL **22**
8.1 Mapa de clasificación . . . 22
8.2 Schema, resolvers y límites . . . 22
8.3 Matriz de autorización GraphQL . . . 23
8.4 Evidencia desde el frontend . . . 23

Módulo 9 Analítica y comunicación gRPC **23**
9.1 Caracterización de la carga . . . 23
9.2 Matriz de necesidades . . . 24
9.3 ADR-01 Servicio de analítica separado . . . 24
9.4 ADR-02 gRPC entre orders e inventory . . . 24
9.5 Detalle del servicio de analítica . . . 25
9.6 Detalle de la comunicación gRPC . . . 25
9.7 Prueba de carga . . . 25
9.8 Pruebas de integración . . . 27

Módulo 10 Backend completo e integración verificable **27**
10.1 Requisitos implementados . . . 27
10.2 Persistencia . . . 27
10.3 Integración de los componentes . . . 28
10.4 Frontend e integraciones externas . . . 28
10.5 Pruebas automatizadas . . . 28
10.6 Matriz de trazabilidad . . . 29
10.7 Demostración de los flujos principales . . . 30
10.8 Inicio, configuración y operación . . . 30
10.9 Limitaciones . . . 30

Anexo A. Modelo C4 **31**

Introducción

Este informe presenta el trabajo que realizamos para el caso N.° 15, Mr. Sandwich, una aplicación de venta de sándwiches, hamburguesas, promociones y bebidas, con opción de retiro o entrega a domicilio. La actividad consiste en diseñar un backend seguro que se comunique con el frontend del proyecto.

Nuestro punto de partida fue una interfaz con menú, detalle de productos y una simulación de inicio de sesión. El desafío fue transformar esas pantallas en un sistema que pudiera consultar y guardar información de verdad, registrar clientes, controlar pedidos y validar los pagos sin confiar en datos modificables desde el navegador.

Objetivo general

Diseñar y documentar un backend para Mr. Sandwich, integrado con el frontend, que considere las funciones del negocio y aplique controles de acceso, separación de servicios y protección de datos.

Objetivos específicos

Identificar los servicios que necesita cada pantalla modelar las entidades y sus relaciones, organizar inventory, orders, clients y payments, definir rutas y permisos, proteger las credenciales y datos personales, y documentar la integración, las pruebas y las limitaciones del proyecto.

Requisitos del caso

Primero identificamos las funciones que debía cumplir la aplicación de Mr. Sandwich. En esta tabla presentamos los requisitos del negocio. Más adelante explicamos las decisiones técnicas que utilizamos para implementarlos.

ID	Requisito
RF01	Ver el menú catálogo por categoría y buscar productos
RF02	Ver el detalle de un producto y personalizarlo
RF03	Registrarse e iniciar o cerrar sesión
RF04	Recuperar la contraseña olvidada
RF05	Armar un pedido con el carrito y conocer el precio oficial antes de confirmar
RF06	Confirmar el pedido para retiro o delivery a una dirección propia
RF07	Pagar el pedido en línea
RF08	Seguir el estado del pedido y consultar el historial
RF09	Administrar perfil y direcciones de entrega
RF10	El Personal preparara y despacha pedidos cambiando su estado
RF11	El Administrador administra productos, precios, disponibilidad y stock
RF12	El Administrador consulta reportes de ventas y alertas de stock bajo
RF13	Anular un pedido no pagado o uno pagado con reembolso

Pantallas

Pantalla	Archivo
P01 Catálogo	index.html
P02 Detalle	producto.html
P03 Login	login.html
P04 Registro	registro.html
P05 Carrito	carrito.html
P06 Seguimiento	pedido.html
P07 Mi cuenta	cuenta.html
P08 Recuperar contraseña	restablecer.html
P09 Panel del local	admin.html
P10 Página de pago	(proveedor)

Brechas declaradas

Estas son las partes del caso que no quedaron completas o que resolvimos con una alternativa de prueba. Las dejamos declaradas para que no se confundan con funciones terminadas. El detalle está en el módulo 10.

Brecha	Requisito	Qué hicimos	Qué falta
Recuperación de contraseña sin envío de correo	RF04	El correo con el enlace se guarda como archivo .eml en la carpeta de ejecución	Conectar un servidor de correo real
Pago con proveedor de prueba	RF07	provider_sim mantiene el contrato del proveedor: sesión de pago, clave de API y webhook firmado	Conectar el entorno de pruebas de un proveedor real
Reembolso no ejecutado	RF13	Un pedido pagado que el administrador anula queda marcado como requiere_reembolso	Ejecutar la devolución del dinero con el proveedor
Vault en modo desarrollo	—	Los secretos se generan en cada arranque y cada servicio solo lee los suyos	Un Vault persistente, hoy los secretos se pierden al detener el sistema
Sin HTTPS entre el navegador y el gateway	—	Todo corre en localhost, la comunicación gRPC interna sí usa mTLS	Certificados HTTPS para el frontend y el API Gateway
Todo en un solo computador	—	La separación pública y privada se hace escuchando en 127.0.0.1 y con run_all.py	Probar el sistema repartido en varias máquinas o contenedores

Módulo 1 Servicios derivados del frontend

En este módulo relacionamos las acciones de cada pantalla con los servicios del backend. Decidimos que el navegador no pueda fijar los precios ni dar por aprobado un pago, el backend comprueba esos datos y calcula el total usando la información oficial de inventario.

1.1 Catálogo de servicios

ID	Requisito	Pantalla y acción	¿Para qué sirve?	Método y ruta de la API	¿Qué recibe?	¿Qué devuelve?	Reglas y posibles errores	Tipo de dato	¿Quién puede usarlo?
S01	RF01	P01: abrir el menú	Muestra los productos del local.	GET /api/v1/products?available&category&q&limit&offset	Filtros opcionales: disponible, categoría, búsqueda (hasta 40 caracteres), límite (1–100) y desde qué resultado empezar.	200: lista de productos, lista vacía si no hay.	400 si los filtros están mal. Se muestran los productos disponibles.	Público	Cualquiera
S01	RF01	P01: elegir categoría	Muestra las categorías del menú.	GET /api/v1/categories	No necesita datos.	200: categorías con ID y nombre.	No se indican errores especiales.	Público	Cualquiera
S02	RF02	P02: abrir un producto	Muestra su detalle, opciones y extras.	GET /api/v1/products/{id}	ID numérico del producto.	200: datos del producto, opciones y extras.	404 si el producto no existe o fue dado de baja.	Público	Cualquiera
S03	RF03	P04: registrarse	Crea una cuenta con el perfil del cliente.	POST /api/v1/auth/register	Nombre (2–60 letras o caracteres), correo válido, contraseña de al menos 8 caracteres con letras y números; teléfono opcional (8 a 12 dígitos, con “+” opcional).	201: ID del usuario y correo.	409 si el correo ya existe 400 si la clave no cumple 503 si falla el sistema de clientes y se revierte el registro.	Credencial y privado	Sin cuenta; con límite de intentos
S04	RF03	P03: iniciar sesión	Comprueba correo y contraseña.	POST /api/v1/auth/login	Correo y contraseña.	200: ID, roles, duración y cookie segura ms_session (HttpOnly, SameSite=Strict).	401 si no coincide, 429 si hubo 5 fallos o demasiados intentos.	Credencial	Sin cuenta, con límite de intentos
S05	RF03	P07/P09: cerrar sesión	Cierra la sesión activa.	POST /api/v1/auth/logout	Cookie de sesión.	200: sesión cerrada y cookie eliminada.	401 si no hay una sesión válida.	Credencial	Usuario con sesión
S06	RF04	P08: pedir enlace	Solicita recuperar la contraseña.	POST /api/v1/auth/password-reset/request	Correo electrónico.	202: responde lo mismo tanto si existe la cuenta como si no.	No revela si un correo está registrado.	Privado	Cualquiera, con límite de intentos
S07	RF04	P08: guardar clave nueva	Cambia la contraseña usando un enlace válido.	POST /api/v1/auth/password-reset/confirm	Token de recuperación y contraseña nueva.	200: clave cambiada.	400 si el token venció o ya se usó, cierra sesiones anteriores.	Credencial	Quien tenga el token de un solo uso
S09	RF09	P07: abrir mi cuenta	Muestra los datos del cliente conectado.	GET /api/v1/clients/me	No necesita datos extra.	200: ID, nombre, correo y teléfono.	401 si no ha iniciado sesión.	Privado	Usuario con sesión
S10	RF09	P07: guardar mi perfil	Actualiza el nombre y el teléfono.	PUT /api/v1/clients/me	Nombre, teléfono opcional.	200: perfil actualizado.	400 si los datos ingresados son incorrectos.	Privado	Usuario con sesión
S11	RF06/RF09	P05/P07: ver direcciones	Lista las direcciones del cliente.	GET /api/v1/clients/me/addresses	No necesita datos extra.	200: lista de sus direcciones.	Solo se consultan las direcciones propias.	Privado	Usuario con sesión
S12	RF09	P07: agregar dirección	Guarda una nueva dirección de entrega.	POST /api/v1/clients/me/addresses	Nombre de la dirección, calle, número, comuna, referencia opcional.	201: dirección creada.	409 si ya tiene 5 direcciones.	Privado	Usuario con sesión
S13	RF09	P07: editar dirección	Cambia una dirección ya guardada.	PUT /api/v1/clients/me/addresses/{id}	ID de dirección y los mismos campos de S12.	200: dirección actualizada.	404 si no es una dirección del usuario.	Privado	Usuario con sesión
S14	RF09	P07: eliminar dirección	Borra una dirección de la cuenta.	DELETE /api/v1/clients/me/addresses/{id}	ID de la dirección.	204: dirección eliminada.	404 si no es propia. Los pedidos anteriores conservan la dirección usada.	Privado	Usuario con sesión
S15	RF05	P05: ver el total	Calcula el precio oficial antes de comprar, sin descontar stock.	POST /api/v1/orders/quote	Productos y cantidades (1–10 por ítem), hasta 20 ítems; opciones y extras opcionales (0–5 por extra).	200: productos, extras, precio por unidad, total de cada ítem y subtotal.	400 si la opción o el extra es inválido; 409 si no está disponible.	Público	Cualquiera

ID	Requisito	Pantalla y acción	¿Para qué sirve?	Método y ruta de la API	¿Qué recibe?	¿Qué devuelve?	Reglas y posibles errores	Tipo de dato	¿Quién puede usarlo?
S16	RF06	P05: confirmar compra	Crea el pedido y reserva los productos.	POST /api/v1/orders + Idempotency-Key	Ítems como S15, retiro o delivery, ID de dirección si corresponde y notas de hasta 140 caracteres. Incluye Idempotency-Key para evitar pedidos duplicados.	201: pedido pendiente de pago, con hora de vencimiento.	409 sin stock o si ya hay más de 3 pedidos pendientes, 404 dirección ajena, 503 si falla inventario: no guarda el pedido.	Privado	Cliente conectado
S17	RF08	P07: ver mis pedidos	Muestra las compras anteriores del cliente.	GET /api/v1/orders/me?limit&offset	Cantidad por página (1–20) y desplazamiento.	200: pedidos y cantidad total.	Solo muestra pedidos del usuario.	Privado	Usuario con sesión
S18	RF08	P06: abrir un pedido	Muestra toda la información de un pedido.	GET /api/v1/orders/{id}	ID del pedido.	200: detalle del pedido.	404 si es de otra persona, no informa si existe.	Privado	Dueño o trabajador autorizado
S19	RF13	P06/P09: anular pedido	Cancela el pedido sin borrar su historial.	POST /api/v1/orders/{id}/cancel	ID del pedido.	200: pedido con estado ANULADO.	409 si el estado no permite anularlo. Libera stock, si estaba pagado, queda marcado para reembolso.	Privado	Dueño, si está pendiente, o administrador
S20	RF10	P09: ver pedidos del local	Muestra los pedidos que atiende el negocio.	GET /api/v1/orders?estado	Filtro de estado opcional.	200: pedidos, total y contacto necesario para atenderlos.	403 si intenta entrar un cliente común.	Privado	Personal o administrador
S21	RF10	P09: cambiar estado	Avanza el pedido en la preparación o entrega.	PUT /api/v1/orders/{id}/status	ID y nuevo estado: en preparacion, listo, en reparto o entregado.	200: pedido actualizado.	409 si el cambio no está permitido. No se acepta pagado como estado desde aquí.	Privado	Personal o administrador
S22	RF07	P05/P06: pagar pedido	Inicia el pago con el proveedor.	POST /api/v1/payments	ID del pedido (order_id).	201: ID del pago, estado, monto y enlace para pagar.	409 si el pedido no espera pago, reutiliza un intento existente, 503 si falla el proveedor.	Privado	Cliente dueño del pedido
S23	RF07/RF08	P06: revisar pago	Consulta el último intento de pago.	GET /api/v1/payments/order/{id}	ID del pedido.	200: información del pago.	404 si no existe o no pertenece al usuario.	Privado	Dueño o personal autorizado
S24	RF07	Proveedor: avisar el pago	Recibe la confirmación de pago del proveedor.	POST /api/v1/payments/webhook	Datos del proveedor y firma con fecha.	200: aviso recibido y resultado indica si era repetido.	401 si la firma no es válida o tiene más de 300 segundos.	Privado	Solo proveedor con firma válida
S25	RF11	P09: ver productos	Muestra productos y existencias del local.	GET /api/v1/admin/products	No necesita datos.	200: productos, stock y mínimo permitido.	403 si entra un cliente común.	Privado del negocio	Personal o administrador
S26	RF11	P09: crear producto	Agrega un producto al catálogo.	POST /api/v1/admin/products	Nombre, categoría, precio entre 100 y 100000, descripción, imagen permitida, disponibilidad, stock y stock mínimo.	201: producto creado.	400 si categoría o imagen no son válidas.	Público y privado	Solo administrador
S27	RF11	P09: editar producto	Cambia los datos de un producto.	PUT /api/v1/admin/products/{id}	ID del producto y los mismos datos que S26.	200: producto actualizado.	404 si el producto no existe.	Público y privado	Solo administrador
S28	RF11	P09: dar de baja	Oculta un producto sin borrarlo del historial.	DELETE /api/v1/admin/products/{id}	ID del producto.	204: producto dado de baja.	404 si no existe, los pedidos antiguos conservan ese producto.	No aplica	Solo administrador
S29	RF11	P09: actualizar stock	Cambia la cantidad disponible.	PUT /api/v1/admin/products/{id}/stock	ID del producto y stock entre 0 y 10000.	200: stock actualizado.	404 si el producto no existe.	Privado del negocio	Personal o administrador

ID	Requisi-to	Pantalla y acción	¿Para qué sirve?	Método y ruta de la API	¿Qué recibe?	¿Qué devuelve?	Reglas y posibles errores	Tipo de dato	¿Quién puede usarlo?
S30	RF08/ RF10/ RF12	P06/P07/P09: ver resúmenes	Reúne datos de varios sistemas en una sola consulta.	POST /api/v1/graphql	Consulta GraphQL	200: datos pedidos y, si corresponde, errores parciales.	Comprueba permisos por consulta y campo, limita consultas demasiado grandes o profundas.	Privado	Usuario con sesión y permisos
S31		Todas: mostrar sesión	Permite saber si hay sesión para mostrar los menús correctos.	GET /api/v1/session	Cookie de sesión, si existe.	200: indica si inició sesión, su ID y roles cuando corresponda.	Solo ayuda a mostrar la interfaz, no da permisos por sí mismo.	Privado	Cualquiera

1.2 Ejemplos de contratos con datos ficticios

S01 GET /api/v1/products?category=bebidas


```
[{"id": 9, "nombre": "Coca-Cola 350 ml", "categoria": "bebidas", "precio": 1990,
"descripcion": "Bebida Coca-Cola en lata, bien helada.", "imagen": "img/bebida.svg", "disponible": true}]
```


S15 POST /api/v1/orders/quote. En este ejemplo mostramos que un precio enviado por el navegador, incluso si fue cambiado intencionalmente, no se utiliza para calcular la cotización del pedido:


```
{"items": [{"product_id": 1, "option_id": "grande", "extras": [{"extra_id": "palta", "qty": 2}], "qty": 2, "precio_
unitario": 1}]}
{"items": [{"product_id": 1, "nombre": "Churrasco italiano", "opcion": "Tamaño grande", "extras": ["Palta extra x2"],
"qty": 2, "precio_unitario": 10090, "total_linea": 20180}], "subtotal": 20180}
```


S16 POST /api/v1/orders con Idempotency-Key: 6f1c…


```
{"items": [{"product_id": 6, "option_id": "doble", "extras": [{"extra_id": "tocino", "qty": 1}], "qty": 1}],
"tipo_entrega": "DELIVERY", "direccion_id": "DIR-a000000001", "notas": "Sin cebolla"}
{"id": "PED-02EBEF86", "estado": "PENDIENTE_PAGO", "tipo_entrega": "DELIVERY",
"items": [{"product_id": 6, "nombre": "Hamburguesa clásica", "opcion": "Carne doble", "extras": ["Tocino extra x1"],
"qty": 1, "precio_unitario": 8380, "total_linea": 8380}],
"subtotal": 8380, "costo_envio": 1990, "total": 10370, "notas": "Sin cebolla",
"creado_en": "2026-10-24T20:35:01Z", "expira_en": "2026-10-24T20:45:01Z",
"direccion": {"id": "DIR-a000000001", "alias": "Casa", "calle": "Calle Falsa", "numero": "123", "comuna": "Santiago",
"referencia": "Depto 4B"},
"historial": [{"estado": "PENDIENTE_PAGO", "en": "2026-10-24T20:35:01Z"}], "requiere_reembolso": false}
```


S22 POST /api/v1/payments: {"order_id": "PED-02EBEF86"}


```
{"payment_id": "PAY-F47E0DEAC2", "order_id": "PED-02EBEF86", "estado": "INICIADO", "monto": 10370,
"comprobante": null, "checkout_url": "http://localhost:8300/checkout/cs_128bf58de413f738"}
```


S24 Webhook. Cabecera X-Provider-Signature: t=1792874000,v1=<hex HMAC-SHA256("t." + cuerpo)>


```
{"id": "evt_3a9c...", "type": "checkout.approved",
"data": {"session_id": "cs_128bf58de413f738", "reference": "PAY-F47E0DEAC2", "amount": 10370}}
```


También utilizamos un formato de error común. Esto permite que el frontend muestre mensajes entendibles sin entregar información interna del servidor:


```
{"error": {"codigo": 404, "mensaje": "Pedido no encontrado"}, "request_id": "349a2a2f8a484c93a57793b8343f3311"}
```


Módulo 2 Clases y cardinalidades del dominio

Para ordenar las clases y sus relaciones preparamos el diagrama UML de dominio. El modelo representa a usuarios, clientes, productos, pedidos, pagos y servicios.

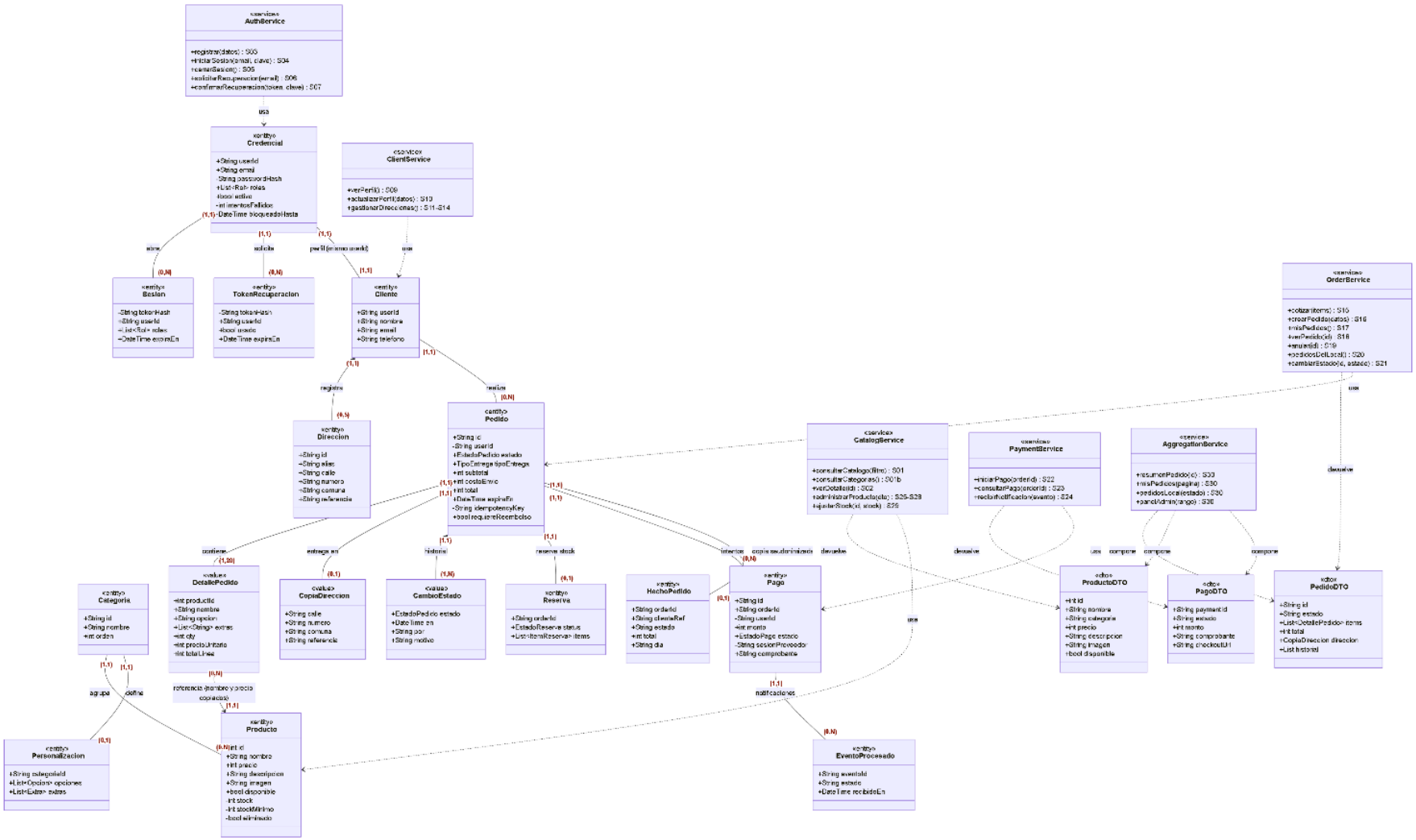


Figura 1: Diagrama UML de clases del dominio y servicios públicos

2.1 Relaciones y reglas

Origen	Destino	Cardinalidad	Regla del negocio que la origina
Credencial	Sesion	1:N	Un usuario puede tener sesiones en varios dispositivos cada sesión es de un solo usuario
Credencial	TokenRecuperacion	1:N	Puede pedir varios enlaces cada token es de un usuario y se usa una sola vez
Credencial	Cliente	1:1	Toda cuenta tiene un perfil,el registro lo crea si falla, se revierte la credencial
Cliente	Direccion	1:N	Composición: la dirección no existe sin su cliente máximo 5 para acotar los datos personales guardados
Categoria	Producto	1:N	Cada producto pertenece a exactamente una categoría del menú
Categoria	Personalizacion	1:1	Las opciones y extras se definen por categoría
Cliente	Pedido	1:N	Un cliente puede no haber comprado nunca cada pedido tiene un dueño
Pedido	DetallePedido	1:N	Un pedido confirmado tiene al menos un ítem el límite de 20 líneas evita abusos. No hay borradores vacíos el carrito no se persiste
Pedido	CopiaDireccion	1:1	Solo los pedidos delivery tienen dirección es una copia, así que borrar o editar la dirección no altera el historial
Pedido	CambioEstado	1:N	Todo pedido nace con el evento pendiente a pago y cada transición agrega un evento
DetallePedido	Producto	N:1	Cada línea referencia un producto, pero copia nombre y precio, si el producto cambia de precio o se da de baja, el historial no cambia
Pedido	Reserva	1:1	Un pedido reserva stock una sola vez siendo idempotente por order_id, si la reserva falla, el pedido no se crea
Pedido	Pago	1:N	Puede haber varios intentos, pero solo uno aprobado con un índice único parcial
Pago	EventoProcesado	1:N	Cada notificación del proveedor se registra una vez siendo clave = id del evento
Pedido	HechoPedido	1:1	La analítica guarda una copia seudonimizada por pedido

2.2 Decisiones del equipo

- Separamos carrito y pedido porque tienen funciones diferentes. El carrito guarda temporalmente productos y cantidades en el navegador, el pedido se crea cuando el servidor comprueba disponibilidad y precio. De esta manera evitamos confiar en montos almacenados por el usuario.
- Pensamos también en lo que pasa si un producto deja de venderse. En ese caso conservamos en el detalle del pedido el nombre y el precio registrados al comprar, de modo que el historial no cambie. El producto se da de baja sin eliminar la información de pedidos anteriores.
- Separamos los datos que el sistema necesita internamente de aquellos que puede mostrar. Por ejemplo, las contraseñas protegidas, el stock interno y las claves de control no aparecen en las respuestas de clientes. La información de contacto de entrega se limita al personal que la necesita.

2.3 Operación UML, ID de servicio

Servicio UML	Operaciones ID
CatalogService	consultarCatalogo a S01, consultarCategorias a S01b, verDetalle a S02, administrarProducto a S25–S28, ajustarStock a S29
AuthService	registrar a S03, iniciarSesion a S04, cerrarSesion a S05, solicitarRecuperacion a S06, confirmarRecuperacion a S07
ClientService	verPerfil a S09, actualizarPerfil a S10, gestionarDirecciones a S11 y S14
OrderService	cotizar a S15, crearPedido a S16, misPedidos a S17, verPedido a S18, anular a S19, pedidosDelLocal a S20, cambiarEstado a S21
PaymentService	iniciarPago a S22, consultarPago a S23, recibirNotificacion a S24
AggregationService	resumenPedido, misPedidos, pedidosLocal, panelAdmin a S30

Módulo 3 Sistemas e integraciones

En esta parte explicamos cómo se conectan los componentes del proyecto. Utilizamos un mapa de arquitectura para mostrar qué sistemas participan y un diagrama de secuencia que sigue una compra con delivery desde el navegador hasta el pago.

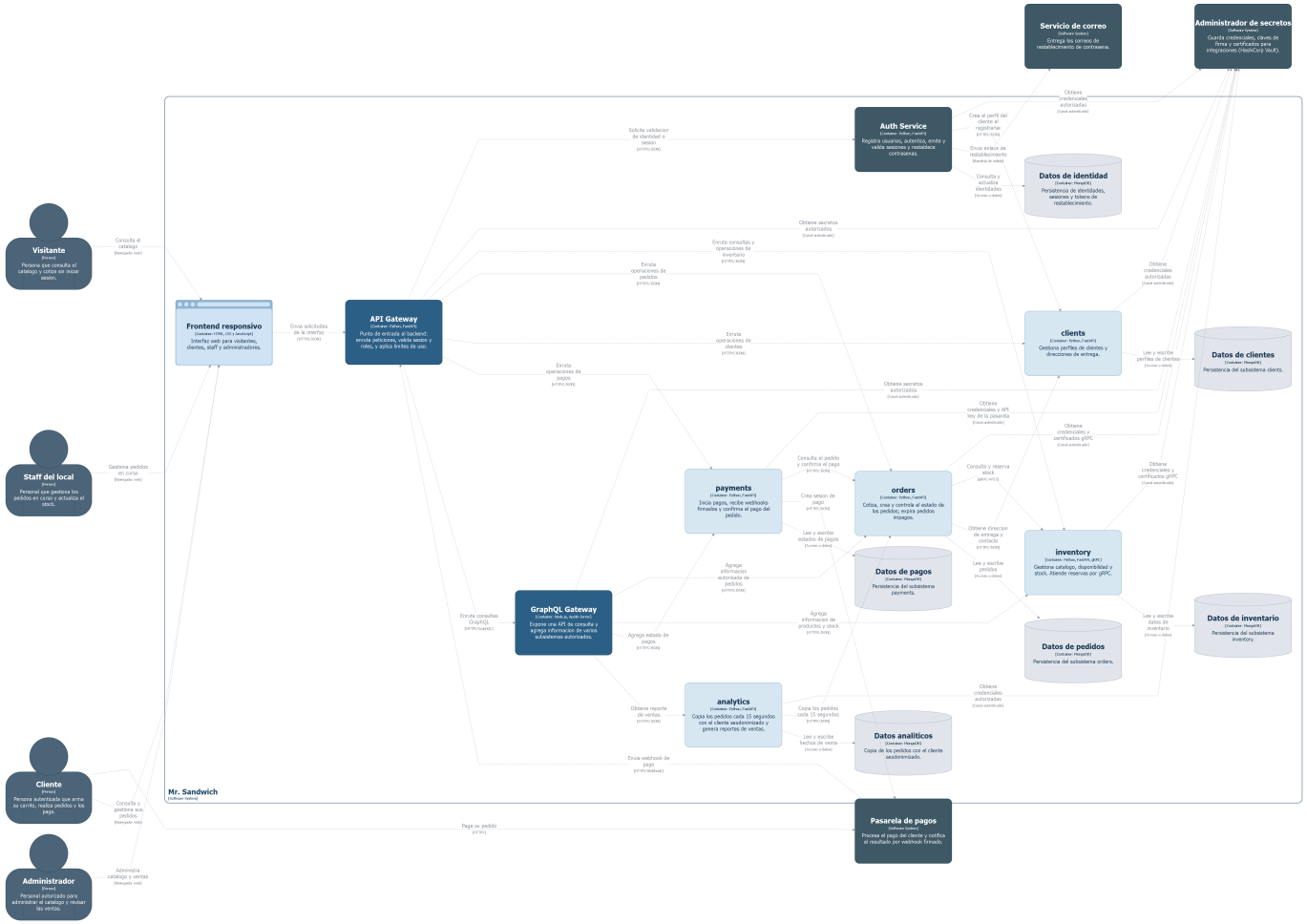


Figura 2: Mapa de componentes

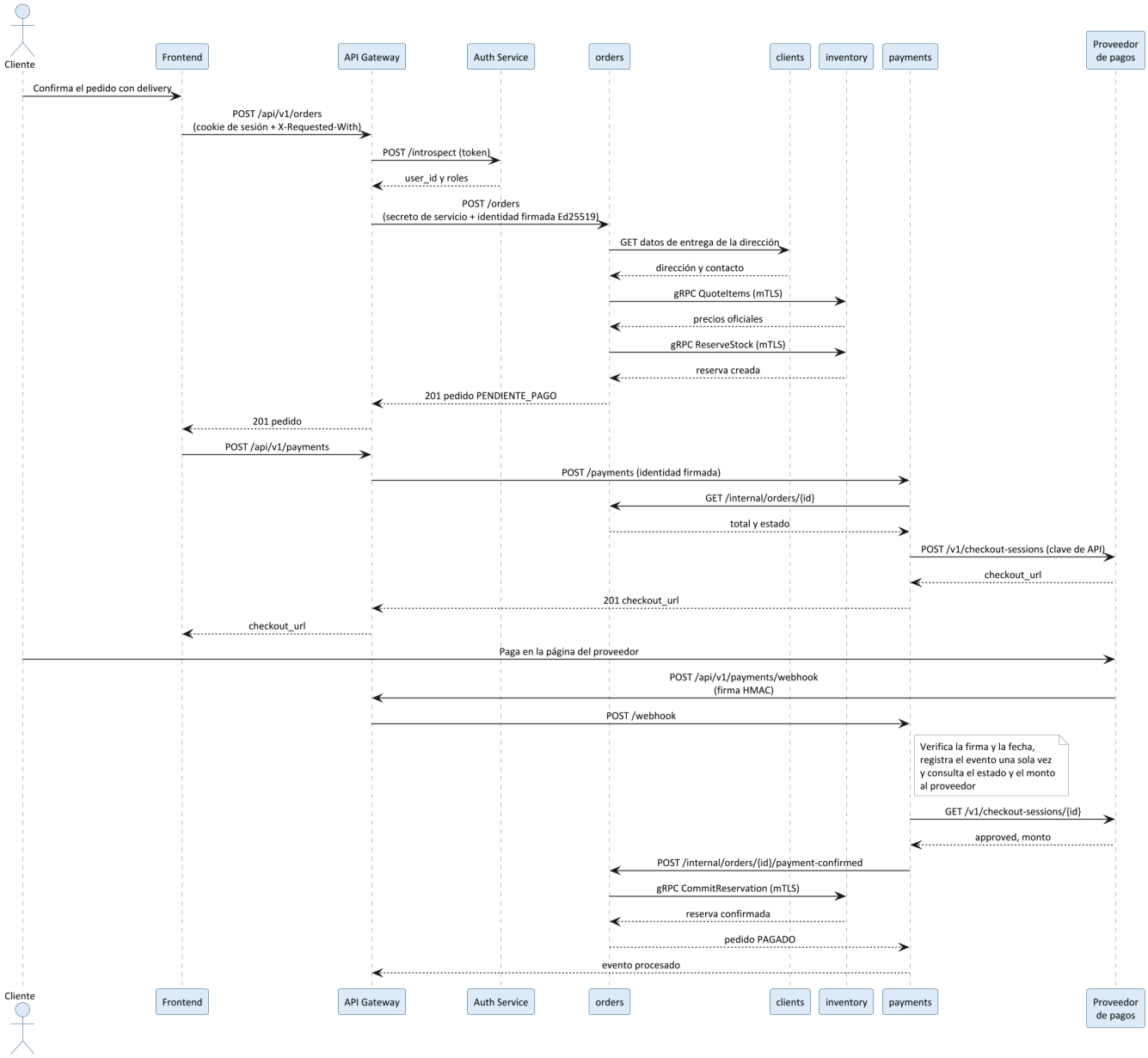


Figura 3: Secuencia de compra con delivery y pago

3.1 Matriz de sistemas

Sistema	Responsabilidad	Datos bajo autoridad	Servicios	Dependen-cias	Contrato de integración	Credencial o identidad	Respuesta ante fallos
inventory	Catálogo, precios, personalización, stock y reservas	products, categories, customization, reservations (ms_inventory). Decide precio y stock	S01, S01b, S02, S15 (vía orders), S25–S29	ninguno	REST (gateway, graphql), gRPC InventoryService (orders)	REST: in_gateway, in_graphql. gRPC certificado de la CA interna con CN=orders	Reserva atómica por producto con reversión, idempotente por order_id
orders	Creación y ciclo de vida del pedido transiciones expiración	orders (ms_orders). Decide el estado del pedido	S15–S21	inventory (gRPC), clients (REST)	REST + gRPC; payment-confirmed interno	in_gateway, in_graphql, in_payments, in_analytics, out_clients,certificado de cliente gRPC	Si inventory no responde 503 y no se guarda nada. Si falla la liberación o confirmación de stock, se marca y la tarea periódica (cada 10 s) reintenta
clients	Perfil y direcciones (datos personales)	clients, addresses (ms_clients). Decide los datos personales	S09–S14	ninguno	REST interno crear perfil (auth), datos de entrega (orders)	in_gateway, in_auth, in_orders	Si falla durante el registro, auth compensa borrando la credencial
payments	Inicio de pagos, verificación con el proveedor, comprobantes	payments, processed_events (ms_payments). Decide el estado del pago y es el único que puede pedir PAGADO	S22–S24	orders, proveedor externo	REST webhook firmado API del proveedor	in_gateway, in_graphql, out_orders, provider_api_key, provider_webhook_secret	Proveedor caído el intento queda FALLIDO y se responde 503. Orders caído en el webhook se responde 503 y se borra el evento para que el proveedor reintente. Pedido ya expirado REQUIERE_REEMBOLSO
auth	Credenciales, sesiones, recuperación	credentials, sessions, reset_tokens (ms_auth)	S03–S07	clients	REST (solo gateway)	in_gateway, out_clients	Bloqueo tras 5 fallos; sesiones con TTL
analytics (	Reportes de ventas	order_facts, etl_state (ms_analytics). Copia, no autoridad	S30 (panelAdmin. ventas)	orders	ETL GET /internal/ orders-export	out_orders, in_graphql	Si orders cae, reintenta en el siguiente ciclo, el reporte informa actualizado_en

Como equipo decidimos separar el backend por responsabilidades, pero sin crear un servicio distinto para cada tabla. Inventory administra productos y stock, orders se encarga de los pedidos, clients gestiona los datos de clientes, y payments verifica los pagos. Esta división facilita entender quién puede modificar cada dato y evita mezclar reglas que cumplen funciones diferentes.

3.2 Intercambios entre sistemas

Origen destino	Operación	Datos mínimos	Autenticación	Errores
orders inventory	gRPC QuoteItems	ids, opción, extras, qty	mTLS (CN=orders)	INVALID_ARGUMENT 400, FAILED_PRECONDITION 409, timeout 503
orders inventory	gRPC ReserveStock , ReleaseReservation , CommitReservation	order_id, [product_id, qty]	mTLS	RESOURCE_EXHAUSTED 409 idempotentes
orders clients	GET /internal/clients/{u}/ delivery-data/{dir}	dirección y contacto (nombre, teléfono)	out_clients	404 si la dirección no es del usuario
auth clients	POST /internal/clients	user_id, nombre, email, teléfono	out_clients	201 se compensa
payments orders	GET /internal/orders/{id}; POST …/payment-confirmed	id, user_id, estado, total payment_id	out_orders	409 pedido no pagable
analytics orders	GET /internal/orders-export? since	sin dirección, contacto ni user_ id; cliente_ref = HMAC	out_orders	Reintenta en el siguiente ciclo
graphql orders/ inventory/payments/ analytics	REST con la identidad reenviada	según el campo pedido	out_+ identidad firmada	Error parcial por campo

Control de stock: Antes de aceptar un pedido, orders solicita a inventory que reserve las unidades necesarias. En el diseño descrito se utiliza una actualización condicional y atómica de MongoDB. Si no hay suficientes productos, se revierten las reservas anteriores, si la comunicación supera el plazo establecido de dos segundos, no se confirma el pedido, esto busca evitar ventas de unidades que ya no están disponibles.

Confirmación del pago: El navegador nunca informa por sí solo que una compra fue pagada. El servicio payments comprueba la firma y la fecha de la notificación, evita procesar un mismo evento dos veces y consulta el estado y el monto al proveedor. Solo después se comunica la aprobación del pago al servicio de pedidos.

Módulo 4 Stakeholders y permisos HTTP

4.1 Registro de stakeholders

Stakeholder	Interés	Rol asociado	Uso de la información
Visitante	Ver el menú y precios	anónimo	Catálogo público
Cliente registrado	Comprar, pagar, seguir pedidos	cliente	Sus propios datos, pedidos y pagos
Personal de cocina y despacho	Preparar y entregar a tiempo	staff	Pedidos en curso con dirección y contacto, ajuste de stock
Administrador del local	Catálogo, precios, ventas	admin	Todo lo anterior más productos, reportes y anulaciones con reembolso
Dueño / gerencia	Rentabilidad	sin permisos operativos	Recibe del administrador los reportes de ventas
Proveedores de insumos	Reposición	sin permisos operativos	El administrador les comunica las alertas de stock bajo por los canales habituales
Repartidor externo	Entregar el pedido	sin permisos operativos	Recibe del personal la dirección y el contacto del pedido asignado
Proveedor de pagos	Cobrar y notificar	identidad técnica provider	Solo S24, autenticado por firma HMAC
Microservicios internos	Colaborar	identidades técnicas gateway, auth, orders, payments, analytics, graphql	Solo las rutas internas listadas en el Módulo 6

4.2 Matriz de permisos

Para interpretar los permisos: P = permitido, D = denegado, / = no corresponde. Además indicamos qué componente realiza la verificación: G = API Gateway, M = microservicio, Q = GraphQL Gateway. Un permiso depende tanto del rol como del pedido o

recurso al que se quiere acceder.

Rol / identidad	Servicio	Ruta	Verbo	Decisión	Alcance y condición	Dónde
Visitante	S01, S01b, S02, S15	/products…, /categories, / orders/quote	GET y POST	P	Solo campos públicos	G
Visitante	S03, S04, S06, S07	/auth/…	POST	P	Límite de 20 por minuto e IP	G
Cliente	S09–S14	/clients/me…	GET/PUT/ POST/DELETE	P	Solo los propios: el user_id sale de la identidad firmada, nunca de la URL	M (clients)
Cliente	S16	/orders	POST	P	Dirección propia, máximo 3 pedidos pendientes	G+M
Cliente	S17, S18	/orders/me, /orders/{id}	GET	P	Solo pedidos propios, un pedido ajeno da 404	M (orders)
Cliente	S19	/orders/{id}/cancel	POST	P	Propio y en pendiente pago	M
Cliente	S21	/orders/{id}/status	PUT	D (403)	/	G
Cliente	S22, S23	/payments…	POST/GET	P	Pedido propio y en pendiente pago	M (payments)
Cliente	S25–S29	/admin/products…	Todos	D (403)	Ver el catálogo no da permisos de administración	G+M
Cliente	S30	/graphql	POST	P	resumenPedido y misPedidos propios, contacto nulo, pedidosLocal y panelAdmin denegados	Q+M
Staff	S18, S20, S21, S23	pedidos y pagos	GET/PUT	P	Todos los pedidos, transiciones válidas según el tipo de entrega	G+M
Staff	S16, S19, S22	crear, anular, pagar	POST	D (403)	El personal no compra ni anula	G/M
Staff	S25, S29	/admin/products, /admin/ products/{id}/stock	GET/PUT	P	Ver productos y ajustar stock	G+M
Staff	S26–S28	crear, editar, dar de baja productos	POST/PUT/ DELETE	D (403)	/	G+M
Admin	S19	/orders/{id}/cancel	POST	P	Pendiente o pagado	M
Admin	S25–S29, S30 panelAdmin	administración y reportes	Todos	P	/	G+M+Q
Cualquiera	marcar PAGADO	/orders/{id}/status	PUT	D	Solo payments, por /internal/…/ payment-confirmed	M
Proveedor	S24	/payments/webhook	POST	P	Firma HMAC válida y reciente	M (payments)
gateway	todas las internas publicadas	/	/	P	Secreto por destino + identidad firmada	M
orders	inventory gRPC	4 métodos	/	P	CN=orders	M (inventory)
payments	orders payment-confirmed, GET /internal/orders/{id}	/	GET/POST	P	/	M
analytics	orders orders-export	/	GET	P	Datos seudonimizados	M

Módulo 5 Clases del API Gateway

Dibujamos un segundo diagrama UML para explicar la organización del API Gateway y sus políticas.

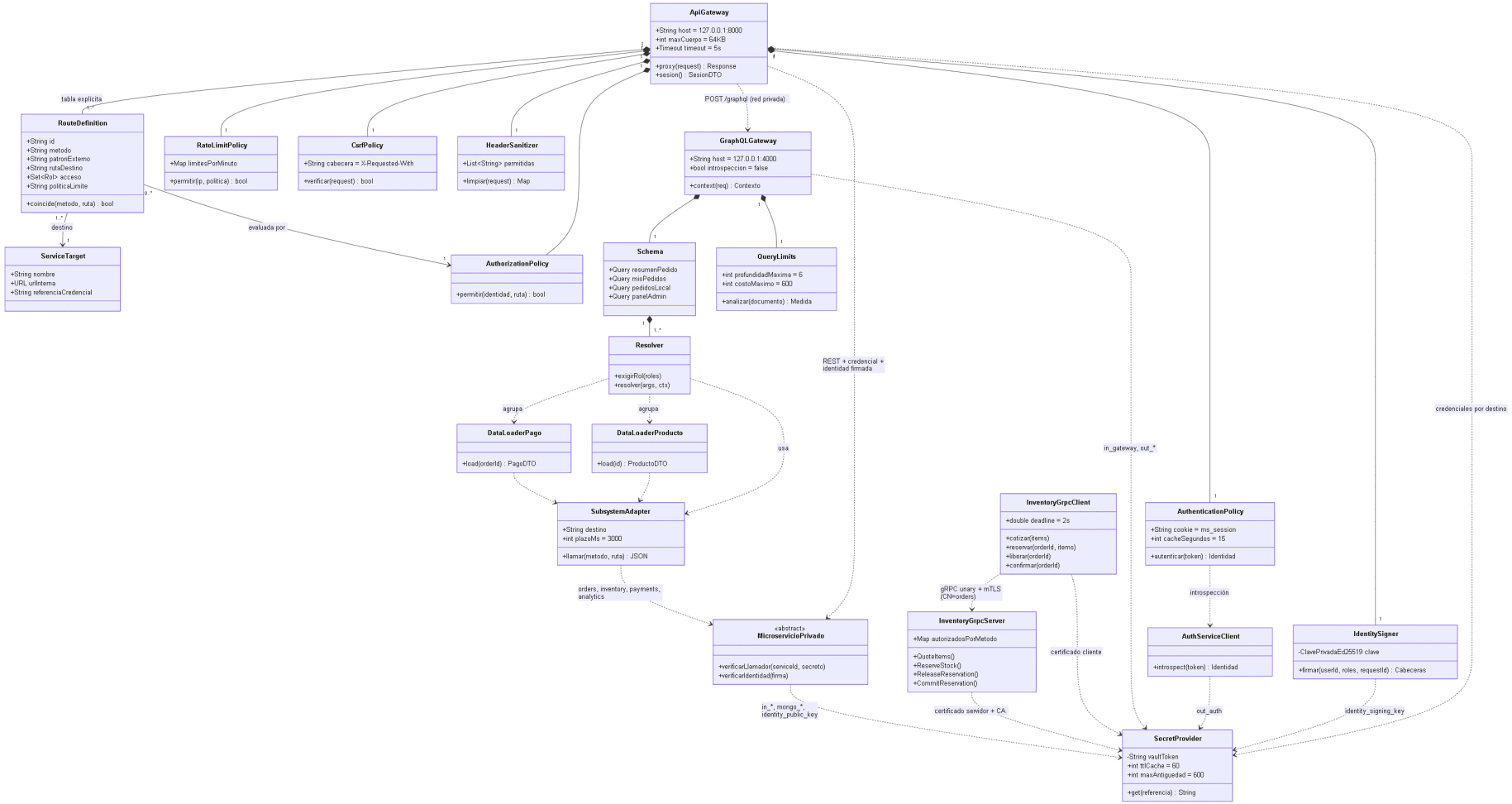


Figura 4: Diagrama UML de clases del API Gateway, políticas, GraphQL Gateway y gRPC

Elegimos un API Gateway porque el frontend necesita un punto de entrada claro. En lugar de que el navegador conozca las direcciones de todos los servicios privados, envía las solicitudes a este componente. El Gateway decide a qué servicio dirigir cada ruta y aplica controles distintos según corresponda a una consulta pública, a un cliente o a un administrador.

Agrupamos las rutas según su acceso. Entre las públicas están el catálogo, el registro y el inicio de sesión. Las rutas de cuenta y pedidos necesitan una sesión válida, las administrativas exigen el rol adecuado. Las rutas internas, como /internal/*, /introspect o /report, no deben quedar expuestas directamente al navegador.

Para comprender la seguridad, identificamos las siguientes fronteras entre componentes:

1. Del navegador al Gateway no se confía automáticamente en lo recibido, se comprueban la sesión y los permisos, se limita el tamaño de la solicitud, se controlan los orígenes y se descartan las cabeceras que no deben venir del cliente.
2. Del Gateway a cada microservicio se usa una credencial propia y una identidad firmada. Con esto los servicios internos pueden verificar quién solicita la operación sin confiar en que el navegador haya escrito correctamente su identidad.
3. Entre orders e inventory se utiliza gRPC con mTLS, de forma que ambos componentes identifican al otro mediante certificados.
4. Cada microservicio usa su propia cuenta de base de datos, con permisos sobre los datos que necesita y no sobre las bases de los demás.

El Gateway hace una primera comprobación de sesión, rol y límites. Sin embargo, cada servicio vuelve a validar la operación, porque allí se conocen las reglas del negocio.

Ejemplo permitido: Cuando Ana consulta un pedido suyo, el Gateway comprueba que inició sesión y envía al servicio orders su identidad verificada. Luego orders compara al usuario con el dueño del pedido. Si coincide, responde con los datos que puede ver.

Ejemplo rechazado: Si Pedro intenta abrir un pedido de Ana, aunque haya iniciado sesión, orders no le permite acceder. Del mismo modo, un cliente que intenta crear productos desde una ruta de administración recibe una respuesta de acceso denegado.

Módulo 6 Rutas y secretos de integración

6.1 Matriz de rutas

En esta tabla seguimos cada solicitud desde la ruta pública hasta el servicio encargado de atenderla. Todos los accesos externos pasan por el Gateway del proyecto. La cookie del usuario se convierte en una identidad interna verificada y cada comunicación utiliza la credencial que le corresponde.

ID	Verbo	Ruta externa	Acceso y rol	Sistema y ruta destino	Controles	Ref. credencial
S01	GET	/api/v1/products	anónimo	inventory GET /products	Validación de filtros, 300/min	Gateway a inventory
S01b	GET	/api/v1/categories	anónimo	inventory GET /categories	300/min	Gateway a inventory
S02	GET	/api/v1/products/{id}	anónimo	inventory GET /products/{id}	Patrón de parámetro	Gateway a inventory
S03	POST	/api/v1/auth/register	anónimo	auth POST /register	20/min, 64 KB	Gateway a auth
S04	POST	/api/v1/auth/login	anónimo	auth POST /login	20/min, emite la cookie	Gateway a auth
S05	POST	/api/v1/auth/logout	autenticado	auth POST /logout (token en el cuerpo)	CSRF, invalida la caché	Gateway a auth
S06/S07	POST	/api/v1/auth/password-reset/*	anónimo	auth /password-reset/*	20/min	Gateway a auth
S09–S14	GET/PUT/POST/DELETE	/api/v1/clients/me[/addresses[/{id}]]	autenticado	clients /me…	CSRF, identidad firmada	Gateway a clients
S15	POST	/api/v1/orders/quote	anónimo	orders POST /quote	300/min	Gateway a orders
S16	POST	/api/v1/orders	cliente	orders POST /orders	CSRF, Idempotency-Key validada	Gateway a orders
S17/S18	GET	/api/v1/orders/me, /api/v1/orders/{id}	autenticado	orders /orders/me, /orders/{id}	Propiedad en orders	Gateway a orders
S19	POST	/api/v1/orders/{id}/cancel	autenticado	orders /orders/{id}/cancel	CSRF	Gateway a orders
S20/S21	GET/PUT	/api/v1/orders, /api/v1/orders/{id}/status	staff, admin	orders /orders, /orders/{id}/status	Rol en G y M	Gateway a orders
S22/S23	POST/GET	/api/v1/payments, /api/v1/payments/order/{id}	cliente / autenticado	payments /payments…	CSRF; propiedad	Gateway a payments
S24	POST	/api/v1/payments/webhook	proveedor	payments POST /webhook	120/min; firma HMAC en payments, sin CSRF	Gateway a payments
S25–S29	GET	/api/v1/admin/products…	staff / admin	inventory /admin/products…	Rol en G y M	Gateway a nventory

ID	Verbo	Ruta externa	Acceso y rol	Sistema y ruta destino	Controles	Ref. credencial
S30	POST	/api/v1/graphql	autenticado	graphql POST /graphql	120/mi, costo y profundidad en Q	Gateway a graphql
S31	GET	/api/v1/session	cualquiera	local en el gateway	/	Gateway a auth

6.2 Operaciones GraphQL

Operación	Tipo	Campos autorizados	Resolver	Subsistemas
resumenPedido	Query	Todos para el dueño excepto contacto, staff y admin, todos	Query.resumenPedido, ResumenPedido.pago, LineaPedido.producto, ResumenPedido.contacto	orders, inventory (lote), payments (lote)
misPedidos	Query	Igual que arriba, solo pedidos propios	Query.misPedidos + DataLoaders	orders, inventory, payments
pedidosLocal	Query	Staff y admin (incluye contacto y direccionEntrega)	Query.pedidosLocal	orders, inventory, payments
panelAdmin(	Query	Solo admin	PanelAdmin.ventas, PanelAdmin. stockBajo	analytics, inventory

6.3 Métodos gRPC

Servicio.método	Entrada y salida	Modali-dad	Identidad permitida	Plazo máximo	Idempotencia
InventoryService. QuoteItems	QuoteRequest y QuoteResponse	unary	CN=orders	2 s	Lectura
InventoryService. ReserveStock	ReserveRequest y ReservationResponse	unary	CN=orders	2 s	Por order_id
InventoryService. ReleaseReservation	ReservationRef y ReservationResponse	unary	CN=orders	2 s	Sí
InventoryService. CommitReservation	ReservationRef y ReservationResponse	unary	CN=orders	2 s	Sí

En el contrato gRPC se establecieron límites de tamaño de mensajes y de cantidad de ítems, además de un número de hilos de atención. El objetivo es evitar solicitudes excesivas que puedan afectar al servicio de inventario.

6.4 Inventario simbólico de secretos e identidades

En este apartado registramos los secretos necesarios para que los servicios se comuniquen. Todos los secretos se generan en scripts/bootstrap.py y se guardan en Vault siendo secret/mrsandwich/consumidor. Cada consumidor recibe un token de Vault propio cuya política solo permite leer su ruta.Cada componente debería acceder solo a la credencial que necesita para cumplir su función.

Referencia	Tipo	Emisor	Consumidor	Finalidad	Ubicación	Alcance	Rotación / revocación
Gateway {auth,clients,inventory, orders,payments,graphql}	Secreto compartido por par	bootstrap	gateway (out_*) / destino (in_gateway)	Autenticar al gateway ante cada servicio	Vault	Un par = un destino	Nuevo en cada arranque, revocar = reiniciar o reescribir solo ese par
Auth a clients, orders a clients, payments a orders, analytics a orders, graphql a {orders, inventory,payments,analytics}	Secreto por par	bootstrap	llamador / destino	Llamadas internas	Vault	Solo las rutas internas de ese par	Cada arranque
identity_signing_key	Clave privada Ed25519	bootstrap	solo gateway	Firmar el contexto de identidad	Vault	Firmar	Cada arranque invalida firmas anteriores, su vigencia es de 60 s
identity_public_key	Clave pública	bootstrap	servicios y graphql	Verificar identidad	Vault	Verificar	Junto con la privada
grpc_ca_cert + grpc_server_* / grpc_client_*	CA interna EC P-256 + certificados (1 día)	bootstrap	inventory (servidor), orders (cliente)	mTLS gRPC	Vault	CN = identidad	Cada arranque, el certificado vence en 24 h
mongo_user / mongo_password por servicio	Credencial de base de datos	bootstrap con el admin de Mongo	Solo su servicio	Acceso a su base	Vault	Una base	La contraseña rota en cada arranque
Admin de MongoDB	Credencial root	bootstrap en primer arranque	Solo bootstrap, seed y run_all stop	Crear y rotar usuarios	~/.mrsandwich/ runtime/ mongo-admin.json fuera de OneDrive y del repo	root	Manual: cambiarla con db. changeUserPassword y actualizar el archivo
provider_api_key	API key del comercio	Proveedor de pagos	payments	Crear y consultar sesiones de pago	Vault	Solo la API del comercio, nunca llega al navegador	La emite el proveedor
provider_webhook_secret	Secreto HMAC	Proveedor de pagos	payments	Verificar notificaciones	Vault	Solo verificar	Como la API key
analytics_pseudonym_key	Clave HMAC	bootstrap	orders	Seudonimizar al cliente en la analítica	Archivo en runtime a Vault	Seudónimos	Estable rotación anual junto con el reinicio de ms_analytics
Token raíz de Vault	Token de administración	run_all	Solo bootstrap	Configurar Vault	Memoria, no se escribe, filtrado de los logs	Total	Se descarta tras el bootstrap
Token pruebas	Token de Vault	bootstrap	Suite de pruebas	Certificado de prueba con CN no autorizado y API key para reenviar webhooks	runtime/pruebas. token	Solo mrsandwich/ pruebas	Cada arranque

Si se descubre una clave expuesta, el procedimiento documentado consiste en revocarla o generar una nueva. En el entorno de desarrollo también se contempla reiniciar los secretos de la instancia.

Si Vault deja de responder, los servicios pueden usar por un plazo limitado la configuración que ya tenían en memoria. Cumplido ese límite, deben detener las operaciones que requieren secretos y devolver un error controlado, en vez de utilizar una contraseña por defecto.

Al comparar nuestro diseño con el material de apoyo del curso, identificamos y corregimos los siguientes problemas mencionados:

- El ejemplo tenía credenciales y valores predeterminados dentro del código, una cabecera compartida por todos los servicios y un proxy demasiado general que permitía más rutas de las necesarias. También propagaba datos de identidad sin firma.
- En nuestra propuesta se reemplazaron esas prácticas por rutas definidas, secretos separados e identidad verificada.

El proyecto se plantea para ejecutarse de manera local durante la evaluación. Al pasar a producción tendríamos que proteger con HTTPS las comunicaciones externas, activar la opción Secure de las cookies y mantener la seguridad entre servicios. El uso de 127.0.0.1 en el laboratorio evita publicar esas direcciones en otras máquinas, pero no reemplaza todas las medidas de seguridad necesarias en producción.

Módulo 7 Datos sensibles y secretos del cliente

7.1 Matriz de datos

Dato o credencial	Finalidad	Clasificación	Custodio	Roles autorizados	Representación en el frontend	Protección	Retención
Producto con nombre, precio, descripción, imagen, disponible	Vender	Público	inventory	Todos	Catálogo y detalle	No es necesario	Mientras exista, baja lógica
Stock exacto, stock mínimo	Operar la cocina	Restringido al negocio	inventory	staff, admin	Solo en P09	No sale en S01/S02	Vigente
Email	Login y contacto	Personal	auth + clients	Dueño	Mi cuenta siendo solo lectura	Índice único, no se registra en logs	Mientras la cuenta exista
Contraseña	Autenticar	Credencial	auth	Nadie la lee	Campo password sin almacenarse	comparación en tiempo constante, nunca se devuelve ni se registra	Hasta que se cambie
Token de sesión	Mantener la sesión	Credencial	auth	Dueño	Cookie HttpOnly	En la base solo el SHA-256, TTL de 30 min, revocable	30 min o hasta el logout
Token de recuperación	Cambiar la clave	Credencial	auth	Dueño del correo	Enlace de un solo uso (se borra de la URL al abrirlo)	SHA-256, 15 min, uso único	TTL
Nombre y teléfono	Despacho	Personal	clients copia en el pedido	Dueño, staff y admin en el pedido	Mi cuenta, P09	Solo por identidad firmada	Mientras la cuenta exista, la copia del pedido sigue la retención del pedido
Dirección	Despacho	Personal	clients copia en el pedido	Dueño, staff y admin en el pedido	Mi cuenta, P05, P06, P09	Solo propias con un máximo 5, nunca en el catálogo ni en la analítica	Clients: hasta que el usuario la borre. Pedido: según la retención contable
Historial de pedidos	Seguimiento y contabilidad	Personal	orders	Dueño, staff, admin	P06, P07	Propiedad verificada en orders, graphql y payments	Contable
API key y secreto de webhook del proveedor	Integración	Secreto de integración	payments	Solo el servicio	Nunca	Vault + política por servicio	Rotación
cliente_ref en analytics	Contar clientes distintos	Seudónimo	analytics	admin	Solo el total en P09	HMAC con clave fuera de analytics	Mientras se use para reportes

7.2 Decisión sobre secretos del cliente

Decidimos no almacenar secretos recuperables de los clientes. La contraseña se guarda mediante un hash y las sesiones también se representan de forma protegida. La aplicación no conserva números de tarjeta ni claves reutilizables. Así diferenciamos una credencial del usuario de una clave que permite integrar servicios.

7.3 Sesiones y tokens

- Creación de sesión. Después de iniciar sesión se emite un token aleatorio, en la base se conserva su hash. Se establece un tiempo de caducidad de 30 minutos para evitar que una sesión pueda utilizarse indefinidamente.
- Transporte. La sesión se envía en una cookie HttpOnly y SameSite=Strict. Estas opciones reducen riesgos como el robo del token mediante JavaScript o las solicitudes desde otros sitios. El backend mantiene además controles complementarios para las acciones que modifican información.
- Comprobación el Gateway consulta al servicio de autenticación para verificar si la sesión sigue activa. La información se mantiene temporalmente en caché y se invalida al cerrar sesión.
- Revocación al cerrar sesión se elimina el acceso, cuando se cambia la contraseña se invalidan las sesiones anteriores. También se limita el ingreso después de varios intentos fallidos.
- Identidad interna el Gateway genera una identidad firmada y con duración corta para que los servicios privados sepan qué usuario originó la solicitud.
- El frontend no contiene claves compartidas ni secretos del proveedor de pagos. Esos elementos quedan en el backend y en el gestor de secretos, no dentro de los archivos HTML o JavaScript que recibe el navegador.

7.4 Recorrido de un dato restringido: la dirección de entrega

1. Primero, el cliente registra una dirección desde “Mi cuenta”. La solicitud pasa por el Gateway y llega a clients, que valida sus campos y relaciona la dirección con el usuario autenticado.
2. Cuando ese cliente compra con delivery, solo selecciona una dirección propia. El sistema comprueba que le pertenece y conserva una copia asociada al pedido para que el historial no cambie si posteriormente edita su perfil.
3. La dirección de entrega solo se muestra a quien compró y al personal autorizado para preparar o despachar el pedido. No se incluye en el catálogo ni en los reportes de ventas. Los registros de diagnóstico deben evitar almacenar direcciones y otros datos personales innecesarios.
4. Si el usuario elimina una dirección de su cuenta, el sistema deja de ofrecerla para pedidos nuevos. La dirección ya utilizada en una compra puede conservarse en el historial mientras exista una justificación de conservación y debe eliminarse cuando corresponda.

7.5 Casos de validación ejecutados

Se ejecutan contra el backend en marcha

Caso	Entrada	Identidad	Resultado esperado	Componente que verifica	Prueba
Acceso anónimo al catálogo	GET /products	ninguna	200, solo campos públicos	gateway + inventory	test_caso1, test_S01_ catalogo_anonimo_solo_ campos_publicos
Operación protegida sin sesión	GET /orders/me, POST /orders, POST /graphql; token inventado	ninguna / falsa	401	gateway	test_caso2_sin_sesion
Token revocado	logout y luego GET /orders/me	Pedro	401	auth	test_caso2_logout_revoca_ token
Administración desde un rol sin permiso	POST/DELETE /admin/ products, GET /orders, panelAdmin	Ana (cliente), Cocina (staff)	403 / FORBIDDEN	gateway + inventory + graphql	test_caso3_rol_sin_permiso
CSRF con la cookie	POST /orders con cookie y sin X-Requested-With	Ana	403	gateway	test_caso3_csrf_cookie_sin_ cabecera
Pedido ajeno	GET /orders/{id}, GraphQL, pago, anulación	Pedro sobre un pedido de Ana	404 / NOT_FOUND, sin datos	orders, payments	test_caso4_pedido_ajeno

Caso	Entrada	Identidad	Resultado esperado	Componente que verifica	Prueba
Campo restringido en GraphQL	resumenPedido{contacto}	Ana / Cocina	Ana: null; Cocina: datos	graphql + orders	test_caso4_campos_restringidos_graphql
Dirección ajena	POST /orders DELIVERY con la dirección de Pedro	Ana	404	clients	test_caso4_direccion_de_otro_cliente
Acceso directo al backend	GET :8103/products, :4000/graphql, etc., sin secreto o con uno inventado	ninguna	403	cada microservicio	test_caso5_acceso_directo_a_servicios_privados
Identidad inyectada	Cabeceras X-Identity (admin) enviadas al gateway	anónima / Ana	401 / 403	gateway	test_caso5_identidad_inyectada_por_el_navegador
Precio alterado	precio_unitario:1, total:1 en S16	Ana	total = 6990	orders + inventory	test_caso6_precio_alterado_se_ignora
Marcar PAGADO desde el frontend	PUT /status PAGADO	Ana / Cocina	403 / 400, sigue PENDIENTE_PAGO	gateway / orders	test_caso6_cliente_no_puede_marcar_pagado
Webhook falsificado	Sin firma, firma con secreto inventado, firma de hace 1 h	ninguna	401 el pedido no cambia	payments	test_caso6_webhook_falsificado
Pago rechazado	Rechazo en la pasarela	Ana	Pago RECHAZADO, el pedido sigue pendiente	payments	test_caso6_pago_rechazado_no_confirma
Logs sin secretos	Revisión de todos los logs	ninguna	Sin claves, tokens ni secretos	todos	test_logs_sin_secretos_ni_credenciales

Módulo 8 Separación privada y agregaciones GraphQL

8.1 Mapa de clasificación

Servicio	Naturaleza	Responsabilidad	Datos propios	Contrato	Dependencias	Justificación de la separación
API Gateway	Borde	Entrada única y políticas	Ninguno	Tabla de rutas	auth, todos	Concentra los controles del borde
Auth	Transaccional	Credenciales y sesiones	ms_auth	REST interno	clients	Las credenciales tienen otra protección y otro ciclo que el perfil
clients	Transaccional	Datos personales	ms_clients	REST	Ninguno	Concentra los datos personales para minimizar quién los toca
inventory	Transaccional + alta demanda	Catálogo y stock	ms_inventory	REST + gRPC	Ninguno	El catálogo es la ruta más consultada, el stock requiere atomicidad
orders	Transaccional	Pedidos	ms_orders	REST + cliente gRPC	inventory, clients	Dueño del estado del pedido
payments	Transaccional	Pagos	ms_payments	REST + webhook	orders, proveedor	Aísla la credencial del proveedor
analytics	Analítica	Reportes	ms_analytics	REST interno	orders	Las agregaciones pesadas no compiten con la base transaccional
GraphQL Gateway	Agregación	Componer vistas	Ninguno	Schema GraphQL	orders, inventory, payments, analytics	No tiene reglas de negocio ni acceso a bases ajenas
Vault	Administrador de secretos	Secretos	En memoria	API de Vault	Ninguno	Sin acceso público, un token por servicio

8.2 Schema, resolvers y límites

- Archivo donde está definido el esquema de GraphQL: backend/graphql_gateway/src/schema.graphql.
- Implementación de los resolvers: src/resolvers.js. Allí se verifica la sesión, el rol y, cuando corresponde, el permiso para ver un campo concreto, como los datos de contacto.
- Para no consultar un producto o pago por separado muchas veces, se agrupan las consultas que pertenecen a una misma solicitud. El diseño usa DataLoaders y rutas internas para obtener los datos de varios pedidos o productos en una sola operación.
- Establecimos límites de tamaño, profundidad y costo de las consultas GraphQL, además de paginación y plazos máximos de respuesta. Estos controles permiten que una pantalla pida los datos necesarios sin ejecutar consultas excesivamente grandes o profundas.
- Si un servicio secundario, como payments o analytics, no responde, GraphQL puede entregar el resto de los datos y marcar únicamente el campo que falló. Esto evita que toda la pantalla quede sin información por un problema parcial.

Consulta GraphQL utilizada como ejemplo para mostrar el resumen de un pedido en P06:

```
query ResumenPedido($id: ID!) {
resumenPedido(id: $id) {
id estado tipoEntrega total
items { nombre cantidad totalLinea producto { imagen disponible } }
pago { estado comprobante }
direccionEntrega { calle numero comuna }
}
}
```

Ejemplo de respuesta con datos ficticios:


```
{"data": {"resumenPedido": {"id": "PED-02EBEF86", "estado": "PAGADO", "tipoEntrega": "DELIVERY", "total": 10370,
"items": [{"nombre": "Hamburguesa clásica", "cantidad": 1, "totalLinea": 8380,
"producto": {"imagen": "img/sliders.webp", "disponible": true}}],
"pago": {"estado": "APROBADO", "comprobante": "CMP-2026-000003"},
"direccionEntrega": {"calle": "Calle Falsa", "numero": "123", "comuna": "Santiago"}}}}
```


8.3 Matriz de autorización GraphQL

Operación / campo	cliente	staff	admin	Verificación
resumenPedido	Solo los propios (otros NOT_ FOUND)	Todos	Todos	orders
misPedidos	Propios	Propios	Propios	orders
pedidosLocal	PROHIBIDO	Si	SI	graphql + orders
panelAdmin	PROHIBIDO	PROHIBIDO	SI	graphql + analytics/inventory
ResumenPedido.contacto	SI	SI	SI	graphql + orders
ResumenPedido.direccionEntrega	Solo la propia	SI	SI	orders
ResumenPedido.pago	Propio	SI	SI	payments repite el filtro de propiedad

8.4 Evidencia desde el frontend

Las páginas P06 (pedido.html), P07 (cuenta.html) y P09 (admin.html) consultan GraphQL a través del API Gateway. La secuencia descrita va desde el catálogo y el carrito hasta el pago simulado y el seguimiento. En esa prueba se modificó un precio en el navegador para comprobar que el servidor no lo aceptaba. El ejemplo visible fue:

"Pedido PED-02EBEF86 Pagado · 1 × Hamburguesa clásica Carne doble Tocino extra x1 Total $10.370 Pago: aprobado comprobante CMP-2026-000003 Delivery a Calle Falsa 123, Santiago (Depto 4B)"

La arquitectura impide que el navegador se comunique directamente con los servicios privados.

Módulo 9 Analítica y comunicación gRPC

En este módulo revisamos si Mr. Sandwich necesita un servicio de analítica separado y si alguna comunicación interna justifica gRPC. Para decidirlo primero describimos la carga del sistema y después dejamos cada decisión en un ADR.

9.1 Caracterización de la carga

Separamos lo que medimos de lo que suponemos. Los valores marcados como **proyección** son supuestos del equipo para un local de sándwiches de tamaño mediano; los marcados como **medición** salen de la prueba de carga de la sección 9.7.

Característica	Valor	Tipo
Volumen del catálogo	11 productos en 4 categorías	Medición
Pedidos por día	Hasta 500	Proyección
Pedidos en la hora punta (almuerzo)	Alrededor de 60	Proyección
Usuarios navegando al mismo tiempo	20	Medición
Tamaño de una cotización	Hasta 20 productos por pedido y 10 unidades por producto, mensaje gRPC limitado a 64 KB	Medición
Frecuencia de cotización	Cada vez que el cliente cambia el carrito	Proyección
Latencia esperada (p95)	Catálogo 200 ms, cotización 300 ms, agregación GraphQL 500 ms	Objetivo del equipo
Datos para reportes	Un registro por pedido, con 500 pedidos al día son unos 15.000 al mes	Proyección

Característica	Valor	Tipo
Necesidad de streaming	No hay flujos continuos: el seguimiento del pedido se consulta	Análisis de requisitos

Con estos números concluimos que los reportes no necesitan big data. Un mes de pedidos cabe sin problema en una colección de MongoDB con índices, y las agregaciones se resuelven en milisegundos.

9.2 Matriz de necesidades

Necesidad	Requisito	¿Aplica?	Decisión	Componente
Reportes de ventas para el administrador	RF12	Sí	Servicio de analítica privado, separado de los pedidos	analytics_service
Cotizar y reservar stock en cada pedido	RF05, RF06	Sí	Comunicación interna gRPC entre orders e inventory	proto/inventory.proto
Mostrar el reporte en una pantalla	RF12, PO9	Sí	El GraphQL Gateway consulta a analytics	panelAdmin.ventas
Procesamiento big data	—	No	La carga proyectada no lo justifica	—
Streaming gRPC	RF08	No	El estado del pedido se consulta, no hay eventos continuos	—
Mensajería asíncrona (por ejemplo STOMP)	—	No	Ningún requisito pide notificaciones	—
Eventos en tiempo real al navegador	RF08	No	La pantalla de seguimiento consulta el pedido	—

9.3 ADR-01 Servicio de analítica separado

El administrador necesita ver ventas por día, productos más vendidos, pedidos por estado y por tipo de entrega (RF12). Si estos reportes se calcularan directo sobre la base de pedidos, una consulta pesada podría hacer más lenta la creación de pedidos, y el reporte tendría acceso a datos personales que no necesita.

Alternativas.

Alternativa	Ventaja	Desventaja
A. Calcular el reporte dentro de orders	Datos siempre al día	Mezcla consultas de reporte con transacciones y expone datos personales
B. Servicio de analítica con copia periódica	Separa la carga y permite quitar datos personales	El reporte tiene un pequeño retraso
C. Plataforma de big data	Escala a volúmenes muy grandes	Excesivo para unos miles de pedidos al mes

Elegimos la alternativa **B**. analytics_service copia los pedidos cada 15 segundos desde orders y calcula los reportes sobre su propia base.

Consecuencias.

- Los reportes no compiten con la creación de pedidos.
- El reporte puede tener hasta unos 15 segundos de retraso, esto lo aceptamos porque el administrador no necesita el dato al segundo. La respuesta incluye actualizado_en para que se vea desde cuándo está actualizado.
- Si orders no responde, el reporte sigue mostrando la última copia y se reintenta en el siguiente ciclo.
- Analytics no es autoridad de ningún dato: si se pierde su base, se reconstruye desde orders.

9.4 ADR-02 gRPC entre orders e inventory

Cada vez que el cliente revisa el carrito o confirma un pedido, orders necesita que inventory calcule el precio oficial y reserve el stock. Es la comunicación interna más frecuente del sistema y tiene reglas estrictas: el precio no puede venir del navegador y una reserva no puede hacerse dos veces.

Alternativas.

Alternativa	Ventaja	Desventaja
A. REST interno con JSON	Igual al resto de los servicios	Contrato sin tipos fijos, los errores se mapean a mano
B. gRPC unary con contrato .proto	Mensajes tipados, deadlines y códigos de error propios	Requiere generar código y manejar certificados

Alternativa	Ventaja	Desventaja
C. gRPC con streaming	Útil para flujos continuos	No hay un flujo continuo: cada cotización es una pregunta y una respuesta

Elegimos la alternativa **B**: cuatro métodos unary siendo QuoteItems, ReserveStock, ReleaseReservation y CommitReservation.

Consecuencias.

- El contrato queda definido en backend/proto/inventory.proto y cualquier cambio se nota al generar el código.
- Usamos mTLS: inventory solo acepta clientes con un certificado de la CA interna, y además revisa que el nombre del certificado sea orders en cada método.
- Cada llamada tiene un plazo de 2 segundos. Si se cumple sin respuesta, orders responde 503 y no guarda el pedido.
- gRPC no se expone al navegador: el frontend solo habla con el API Gateway por HTTP.
- No usamos streaming porque ningún flujo lo necesita.

9.5 Detalle del servicio de analítica

Aspecto	Cómo lo resolvimos
Origen de datos	GET /internal/orders-export de orders, que solo acepta llamadas de analytics
Actualización	Copia incremental cada 15 segundos, en lotes de hasta 500 pedidos, guardando la marca del último pedido copiado
Retraso aceptado	Hasta unos 15 segundos más el tiempo de la copia
Agregaciones	Total vendido, pedidos pagados, ticket promedio, clientes distintos, ventas por día, productos más vendidos, pedidos por estado y por tipo de entrega
Permisos	Solo el GraphQL Gateway puede llamar a analytics, y solo con la identidad de un administrador
Datos personales	La copia no incluye dirección, teléfono, nombre ni user_id. Cada cliente se representa con un código (cliente_ref) calculado con HMAC, que permite contar clientes distintos sin saber quiénes son
Consulta desde la pantalla	El panel del local (P09) pide panelAdmin { ventas } al GraphQL Gateway, que llama a /reports/sales

9.6 Detalle de la comunicación gRPC

Aspecto	Cómo lo resolvimos
Contrato	backend/proto/inventory.proto, con mensajes tipados para cada método
Modalidad	Unary en los cuatro métodos
Autenticación del servicio	mTLS con certificados emitidos por una CA interna al arrancar el sistema
Autorización por método	Cada método tiene una lista de servicios permitidos; hoy solo orders
Deadline	2 segundos por llamada
Cancelación	Si se cumple el plazo, la llamada se cancela y orders responde 503
Límites	Máximo 20 productos por cotización, 10 unidades por producto y mensajes de 64 KB
Errores	INVALID_ARGUMENT → 400, NOT_FOUND → 404, FAILED_PRECONDITION y RESOURCE_EXHAUSTED → 409, sin respuesta → 503
Idempotencia	Las reservas usan el order_id como clave: repetir una reserva no descuenta el stock dos veces

9.7 Prueba de carga

Tipo: medición, no proyección.

Entorno: Windows 11, 12 núcleos lógicos, 31,7 GB de RAM, Python 3.12.3. Todos los componentes y el generador de carga corren en el mismo computador, así que compiten por la CPU y los resultados son conservadores.

Datos: los datos de prueba de scripts/seed_data.json.

Comando: python scripts/prueba_carga.py –concurrencia 20 –duracion 30 –etiqueta estres, con el sistema iniciado con python run_all.py –reset –perfil carga. El perfil de carga solo desactiva los límites de solicitudes del gateway, para medir los servicios y no el límite.

Escenario	Concurrencia	Duración	Solicitudes	Throughput	p95	Objetivo p95	Errores	CPU prom./máx.	Memoria máx.	Resultado
E1 Catálogo	20	30 s	28.012	934,6 req/s	33,1 ms	200 ms	0 %	217 % / 251 %	890 MB	Cumple
E2 Cotización (gRPC)	20	30 s	21.790	726,6 req/s	40,7 ms	300 ms	0 %	349 % / 388 %	897 MB	Cumple
E3 Agregación GraphQL	20	30 s	18.023	601,2 req/s	49,8 ms	500 ms	0 %	301 % / 352 %	1.038 MB	Cumple

La CPU es la suma de los procesos del backend y la memoria es la suma de lo que usan esos procesos.

Comparación con los objetivos. Los tres escenarios cumplen el objetivo de p95 con bastante margen y sin errores. Aun así no afirmamos que gRPC sea más rápido que REST, porque no medimos una versión REST de la misma operación. Lo que sí muestra la prueba es que la cotización por gRPC, que es la operación más frecuente, se mantiene muy por debajo del objetivo.

Los resultados completos están en docs/evidencias/prueba_carga_estres.json.

9.8 Pruebas de integración

Prueba	Qué comprueba	Resultado
test_grpc_rechaza_sin_certificado_cliente	inventory rechaza conexiones sin certificado	Pasó
test_grpc_rechaza_certificado_de_ca_no_confiable	inventory rechaza certificados de otra CA	Pasó
test_grpc_autorizacion_por_metodo	un servicio que no es orders no puede llamar a los métodos	Pasó
test_inventory_no_responde_pedido_no_se_crea	si inventory no responde, el pedido no se guarda	Pasó
test_reserva_atomica_entre_productos	si falta stock de un producto, se revierte la reserva de los demás	Pasó
test_analitica_refleja_ventas	un pedido pagado aparece en el reporte	Pasó
test_analytics_no_responde_error_parcial	si analytics cae, GraphQL devuelve el resto con un error parcial	Pasó
test_agregacion_combina_tres_subsistemas	GraphQL combina orders, inventory y payments	Pasó

La salida completa de las pruebas está en docs/evidencias/pruebas_pytest.txt.

Módulo 10 Backend completo e integración verificable

En este módulo mostramos que el sistema funciona completo y que los controles de seguridad se cumplen mientras corre, no solo en el papel. Todo lo que aparece acá se ejecutó en un computador del equipo y sus resultados están en docs/evidencias.

10.1 Requisitos implementados

Requisito	Estado	Observación
RF01 Ver el menú y buscar	Implementado	La búsqueda trata el texto como texto, no como expresión regular
RF02 Detalle y personalización	Implementado	Opciones y extras vienen de inventory
RF03 Registro, inicio y cierre de sesión	Implementado	Cerrar sesión revoca el token en el servidor
RF04 Recuperar contraseña	Implementado con limitación	El correo se guarda como archivo .eml en ~/.mrsandwich/runtime/outbox en vez de enviarse
RF05 Carrito y precio oficial	Implementado	El precio lo calcula inventory, lo que mande el navegador se ignora
RF06 Confirmar pedido con retiro o delivery	Implementado	La dirección debe ser del mismo cliente
RF07 Pagar en línea	Implementado con proveedor de prueba	Ver 10.4
RF08 Seguir el pedido e historial	Implementado	Solo el dueño y el personal ven el pedido
RF09 Perfil y direcciones	Implementado	
RF10 Preparar y despachar	Implementado	Solo se permiten las transiciones válidas
RF11 Administrar productos y stock	Implementado	Los productos se dan de baja sin borrar el historial
RF12 Reportes y stock bajo	Implementado	Mediante analytics y GraphQL
RF13 Anular pedidos	Implementado con limitación	El administrador puede anular un pedido pagado y queda marcado requiere_reembolso, la devolución del dinero no se ejecuta porque el proveedor de prueba no tiene esa operación

10.2 Persistencia

Aspecto	Cómo lo resolvimos
Motor	MongoDB 9.0 con autenticación activada
Separación	Una base por servicio: ms_auth, ms_clients, ms_inventory, ms_orders, ms_payments y ms_analytics
Usuarios de base de datos	Cada servicio tiene su propio usuario, que solo puede leer y escribir su base. Las claves se generan en cada arranque y se guardan en Vault
Esquema e índices	Cada servicio crea sus índices al iniciar
Scripts de creación	scripts/bootstrap.py crea los usuarios, los certificados y los secretos; scripts/seed.py carga los datos de prueba

Aspecto	Cómo lo resolvimos
Datos de prueba	scripts/seed_data.json: 4 categorías, 11 productos y 4 usuarios ficticios
Validaciones	Los datos que llegan se validan en el servidor con modelos Pydantic
Control de estados	Los pedidos solo cambian de estado con actualizaciones condicionales, así dos cambios simultáneos no se pisan
Errores	Todas las respuestas de error tienen el mismo formato {"error": {...}, "request_id": ...} y no muestran detalles internos

10.3 Integración de los componentes

Componente	Puerto	Expuesto al navegador	Cómo se protege
Frontend	8080	Sí	Solo archivos estáticos, no guarda secretos
API Gateway	8000	Sí	Rutas declaradas, permisos por rol, límites de solicitudes y cabeceras anti-CSRF
Auth Service	8101	No	Solo acepta llamadas del gateway
Microservicios	8102–8106	No	Cada uno solo acepta a los servicios autorizados, con un secreto por pareja de servicios
GraphQL Gateway	4000	No	Solo acepta al API Gateway
Inventory gRPC	50051	No	mTLS con certificados de la CA interna
Vault	8200	No	Cada servicio tiene un token que solo puede leer sus propios secretos
MongoDB	27018	No	Usuario y clave por servicio

Todos los componentes escuchan en 127.0.0.1. La identidad del usuario la verifica el gateway y la reenvía firmada y ademas los servicios internos no confían en cabeceras enviadas por el navegador. Ningún secreto está escrito en el código: todos se generan al arrancar y se leen desde Vault.

Para demostrar la separación entre la parte pública y la privada usamos run_all.py, que es el entorno reproducible del proyecto. No usamos contenedores porque lo visto en clase con Vault funcionó como ejecutable, y run_all.py deja el mismo resultado: solo el frontend y el gateway quedan pensados para el navegador.

10.4 Frontend e integraciones externas

El frontend llama solo al API Gateway (frontend/js/api.js). No hay datos escritos en el frontend: el catálogo, los pedidos, el perfil y los reportes vienen de las APIs reales.

Integración externa	Situación	Cómo la resolvimos
Proveedor de pagos	No usamos un proveedor real	provider_sim es un adaptador de prueba que mantiene el contrato: sesión de pago, autenticación con clave de API, webhook firmado con HMAC y estados aprobado o rechazado
Correo electrónico	No hay servidor de correo	Los correos de recuperación se guardan como archivos .eml

El adaptador de pagos no reemplaza a payments: nuestro servicio sigue verificando la firma, consultando el estado al proveedor y evitando pagos duplicados.

10.5 Pruebas automatizadas

Ejecutamos 51 pruebas con el sistema corriendo y todas pasaron (docs/evidencias/pruebas_pytest.txt y docs/evidencias/prueba_expiracion.txt). Además, el GraphQL Gateway tiene 5 pruebas propias (npm test).

Grupo	Pruebas	Resultado
Contratos (test_01_contratos.py)	11	11 pasaron
Seguridad (test_02_seguridad.py)	15	15 pasaron
Reglas de negocio (test_03_negocio.py)	11	11 pasaron
GraphQL, gRPC y caídas (test_04_graphql_grpc_resiliencia.py)	13	13 pasaron
Límite de solicitudes (test_99_limite_solicitudes.py)	1	1 pasó
Límites del GraphQL Gateway (npm test)	5	5 pasaron

Lo que pide revisar la pauta y la prueba que lo cubre:

Control	Prueba
Propiedad de recursos	test_caso4_pedido_ajeno, test_caso4_direccion_de_otro_cliente
Roles	test_caso3_rol_sin_permiso
Rechazo del acceso directo	test_caso5_acceso_directo_a_servicios_privados, test_caso5_identidad_inyectada_por_el_navegador
Límites GraphQL	test_paginacion_limitada, test_limite_de_costo, test_introspeccion_y_lotes_deshabilitados
Caída de dependencias	test_inventory_no_responde_pedido_no_se_crea, test_analytics_no_responde_error_parcial, test_payments_no_responde_resumen_parcial, test_proveedor_de_pagos_no_responde, test_gestor_de_secretos_no_responde_usa_cache
Pagos duplicados	test_pago_duplicado_y_notificacion_repetida, test_idempotency_key_evita_pedidos_duplicados
Recuperación de stock	test_anulacion_recupera_stock, test_expiracion_libera_stock
gRPC	test_grpc_rechaza_sin_certificado_cliente, test_grpc_rechaza_certificado_de_ca_no_confiable, test_grpc_autorizacion_por_metodo
Analítica	test_analitica_refleja_ventas

10.6 Matriz de trazabilidad

Requisito	Pantalla	Servicio	Prueba	Resultado	Evidencia
RF01	P01 index.html	S01, S01b	test_S01_catalogo_anonimo_solo_campos_publicos, test_S01_filtros, test_S01_busqueda_no_interpreta_regex	Pasó	pruebas_pytest.txt, demostración paso 1
RF02	P02 producto.html	S02	test_S02_detalle_con_personalizacion	Pasó	pruebas_pytest.txt; demostración paso 1
RF03	P03 login.html, P04 registro.html	S03, S04, S05	test_S03_registro_y_perfil, test_S03_politica_de_contrasena, test_caso2_logout_revoca_token	Pasó	pruebas_pytest.txt; demostración pasos 3 y 12
RF04	P08 restablecer.html	S06, S07	test_S06_S07_recuperacion_de_contrasena	Pasó	pruebas_pytest.txt
RF05	P05 carrito.html	S15	test_S15_cotizacion_calcula_en_servidor, test_caso6_precio_alterado_se_ignora	Pasó	pruebas_pytest.txt; demostración paso 4
RF06	P05 carrito.html	S16	test_flujo_completo_delivery, test_stock_insuficiente_no_crea_pedido, test_reserva_atomica_entre_productos, test_idempotency_key_evita_pedidos_duplicados	Pasó	pruebas_pytest.txt; demostración paso 5
RF07	P06 pedido.html, P10 página de pago	S22, S23, S24	test_caso6_cliente_no_puede_marcar_pagado, test_caso6_webhook_falsificado, test_caso6_pago_rechazado_no_confirma, test_pago_duplicado_y_notificacion_repetida	Pasó	pruebas_pytest.txt; demostración paso 6
RF08	P06 pedido.html, P07 cuenta.html	S17, S18 ; resumenPedido	test_caso4_pedido_ajeno, test_agregacion_combina_tres_subsistemas, test_caso2_sin_sesion	Pasó	pruebas_pytest.txt; demostración pasos 2 y 7
RF09	P07 cuenta.html	S09–S14	test_S11_S14_direcciones_crud, test_caso4_direccion_de_otro_cliente	Pasó	pruebas_pytest.txt
RF10	P09 admin.html	S20, S21	test_flujo_completo_delivery, test_transiciones_invalidas	Pasó	pruebas_pytest.txt; demostración paso 8
RF11	P09 admin.html	S25–S29	test_caso3_rol_sin_permiso, test_baja_logica_de_producto_conserva_historial	Pasó	pruebas_pytest.txt; demostración paso 10
RF12	P09 admin.html	S30 panelAdmin	test_analitica_refleja_ventas, test_analytics_no_responde_error_parcial	Pasó	pruebas_pytest.txt; demostración paso 11

Requisito	Pantalla	Servicio	Prueba	Resultado	Evidencia
RF13	P07 cuenta.html, P09 admin.html	S19	test_anulacion_recupera_stock, test_expiracion_libera_stock	Pasó	pruebas_pytest.txt, prueba_expiracion.txt

10.7 Demostración de los flujos principales

Además de las pruebas, ejecutamos los flujos principales uno por uno contra el sistema real y guardamos la salida en docs/evidencias/demostracion_flujos.md. Cada línea muestra quién hace la solicitud, el código de respuesta y el request_id con que quedó registrada en los logs.

El caso que pide la pauta como ejemplo es el paso 7. Ana crea un pedido y lo consulta sin problema, Pedro, con otra cuenta, pide el mismo pedido y recibe 404 Pedido no encontrado, y la respuesta no trae ningún dato del pedido. Por GraphQL pasa lo mismo: resumenPedido devuelve null con el error NOT_FOUND. Las dos respuestas salen de la misma regla, la función puede_ver de orders, que deja ver un pedido solo a su dueño o al personal. Por eso el personal de cocina sí lo puede ver: lo necesita para despacharlo.

Paso	Flujo	Resultado
1	Visitante ve el catálogo	200, solo campos públicos
2	Visitante pide sus pedidos sin sesión	401
3	Inicio de sesión de los cuatro usuarios	200
4	Cotización enviando un precio falso	El servidor ignora el precio y calcula el oficial
5	Pedido con delivery	201, queda pendiente de pago
6	Pago con el proveedor de prueba	Pago aprobado con comprobante y pedido pagado
7	Pedido propio y pedido ajeno	200 para la dueña, 404 sin datos para otra cuenta
8	El personal despacha y un cliente intenta cambiar el estado	200 para el personal, 403 para el cliente
9	Llamar directo a orders sin pasar por el gateway	403
10	Un cliente intenta crear un producto	403
11	Reporte de ventas del administrador	Aparece la venta; un cliente recibe FORBIDDEN
12	Cierre de sesión	El token anterior pasa a dar 401

10.8 Inicio, configuración y operación

Lo que pide la pauta	Dónde está
Iniciar y detener	README.md: python run_all.py –reset y python run_all.py stop
Configurar secretos	Los genera run_all.py en cada arranque y los guarda en Vault, ningún archivo del repositorio tiene valores reales
Variables de configuración	.env.example, con valores de ejemplo y VAULT_TOKEN vacío
Crear datos de prueba	python run_all.py –reset ejecuta scripts/seed.py
Ejecutar las pruebas	python -m pytest -v y npm test en backend/graphql_gateway
Dependencias versionadas	requirements.txt, requirements-dev.txt y backend/graphql_gateway/package-lock.json
Health checks	Cada servicio responde GET /health, run_all.py espera esa respuesta antes de seguir
Logs con identificador	Cada solicitud lleva un X-Request-ID que pasa por todos los servicios. Los logs no guardan tokens, claves ni datos personales

10.9 Limitaciones

- El sistema corre completo en un solo computador, no lo probamos repartido en varias máquinas.
- Vault funciona en modo desarrollo y en memoria: al detener el sistema se pierden los secretos y se generan otros al volver a arrancar. Para producción habría que usar un Vault persistente.
- Los pagos y los correos usan adaptadores de prueba.
- La devolución del dinero de un pedido anulado queda marcada, pero no se ejecuta.
- No usamos HTTPS entre el navegador y el gateway porque todo corre en localhost, la comunicación gRPC interna sí va cifrada con mTLS.

Anexo A. Modelo C4

Los diagramas de este anexo se generan desde un solo modelo escrito en Structurizr DSL (docs/diagramas/workspace.dsl), la herramienta vista en clases para el modelo C4. El archivo es editable: se puede abrir en https://playground.structurizr.com y cualquier cambio se refleja en todas las vistas.

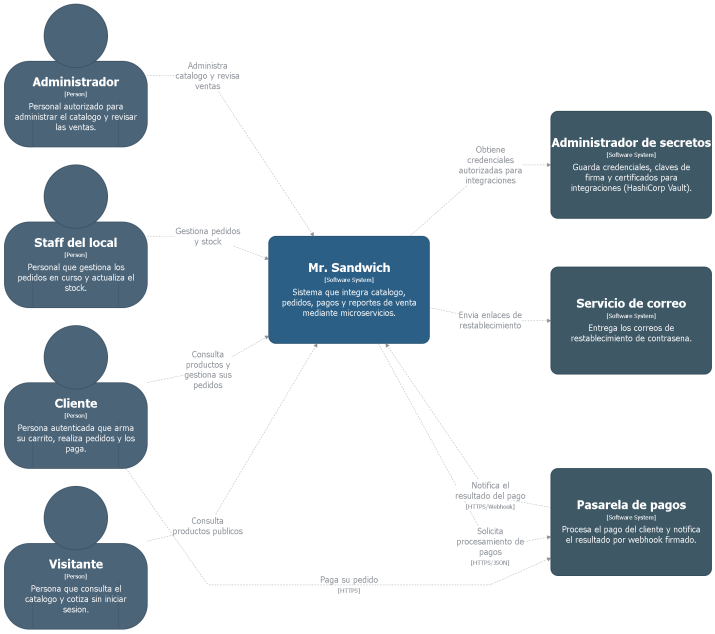


Figura 5: C1, contexto de Mr. Sandwich

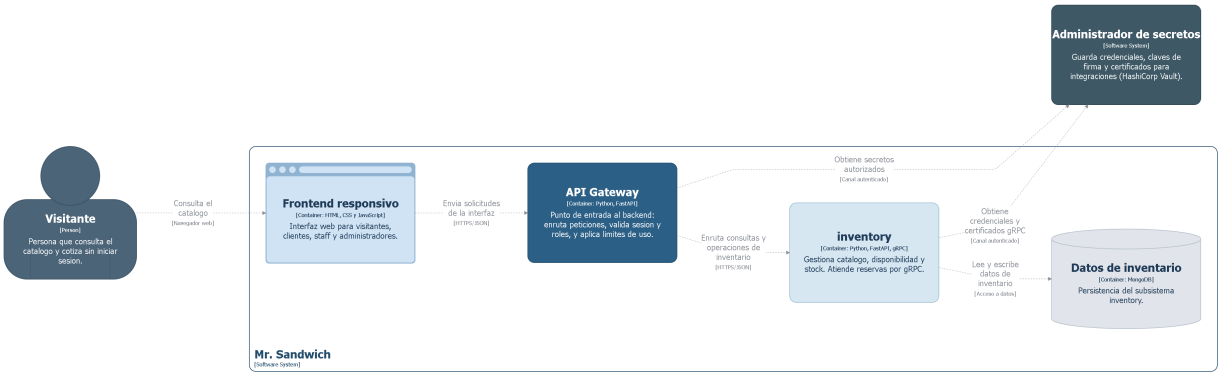


Figura 6: C2, contenedores que participan en la consulta del catálogo

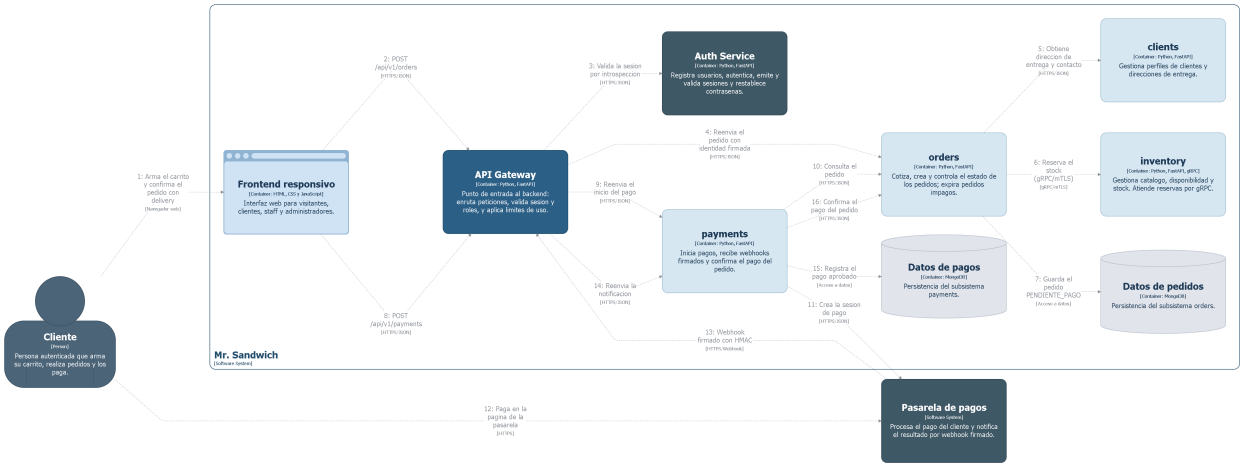


Figura 7: C2, secuencia numerada del registro y pago de un pedido

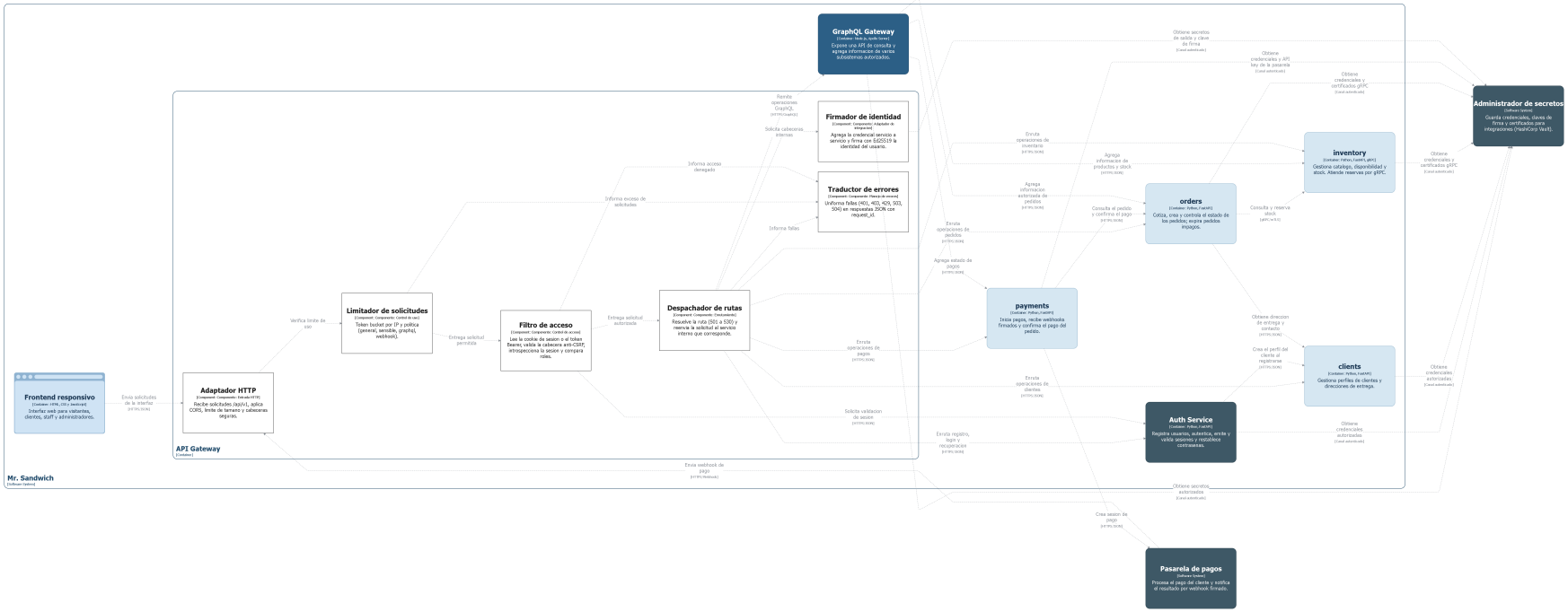


Figura 8: C3, componentes internos del API Gateway

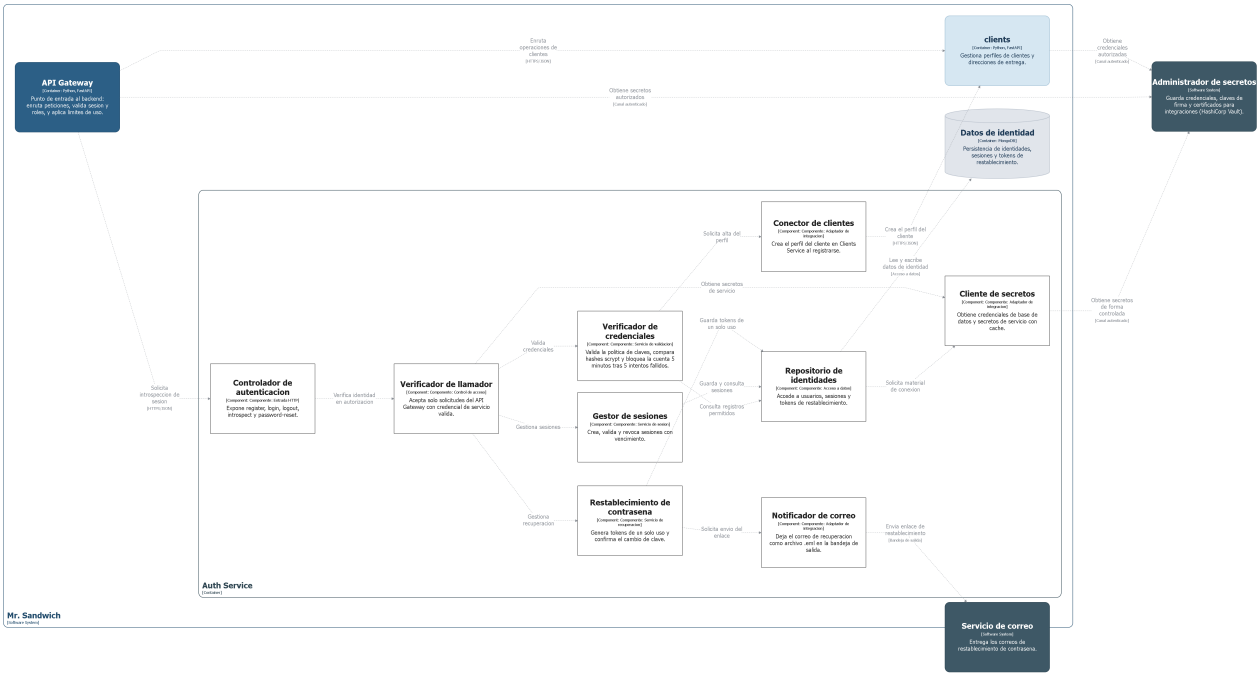


Figura 9: C3, componentes internos del Auth Service

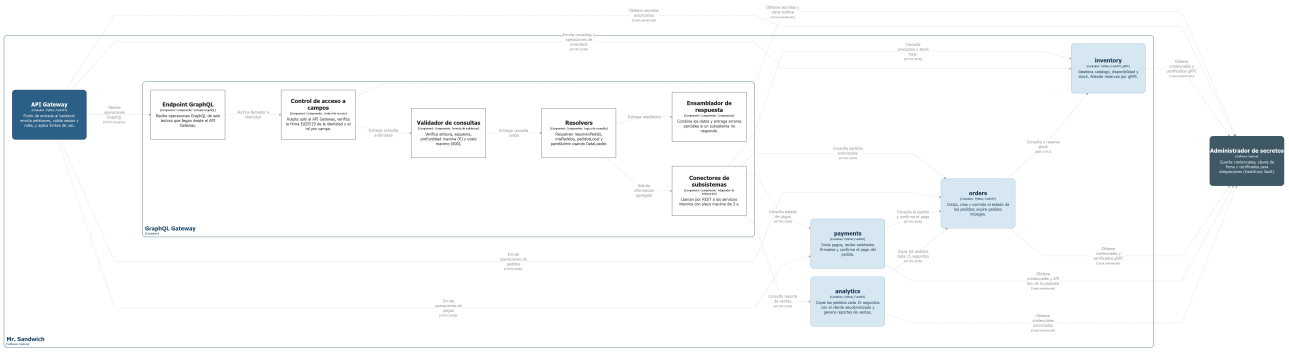


Figura 10: C3, componentes internos del GraphQL Gateway